

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Flood Situation Monitoring and Reporting System

By
Do Nguyen Chuong - ITITIU22021

*A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Computer Science*

Ho Chi Minh City, Vietnam
2026

Flood Situation Monitoring and Reporting System

APPROVED BY:

Le Duy Tan, Ph.D., supervisor

THESIS COMMITTEE

Le Duy Tan, Ph.D

.....

Acknowledgments

I would like to thank my thesis supervisor, Dr. Le Duy Tan, for his guidance throughout this work. His comments helped me narrow the topic, connect the system design with the actual source code, and keep the report focused when the project started to become too wide. I am especially grateful for the time he spent reviewing my ideas and pointing out where the work still needed to be clearer.

I also want to thank the lecturers at the School of Computer Science and Engineering. The courses and discussions I had there gave me the background I needed to build VietFlood across backend services, a mobile app, a web dashboard, and deployment tools.

I am thankful to the staff at International University as well. Their work behind the scenes made my study process smoother than it would have been otherwise.

My classmates and colleagues also helped more than they may realize. Some helped me talk through technical problems. Some simply listened when the project felt messy. Those small conversations made the work easier to continue.

Finally, I would like to thank my family for their patience and support during my studies. Their trust in me gave me room to finish this thesis.

This thesis carries the help of many people. I am grateful to all of them.

Table of Contents

List of Tables	vii
List of Figures	ix
List of Abbreviations	x
Abstract	xi
1 Introduction	1
1.1 The necessity of flood reporting and relief coordination by information technology	1
1.1.1 What is flood reporting and relief coordination by technology? . . .	1
1.1.2 Some approaches to supporting flood reporting and relief coordination using technology	2
1.2 Problem Statement	2
1.3 Scope And Objectives	3
1.3.1 Goals of VietFlood	4
1.3.2 System Actors and Basic Functions	4
1.3.3 Thesis Structure	5
2 Background and Related Works	6
2.1 Background	6
2.1.1 Flood Reports as Structured Field Records	6
2.1.2 The Role of Mobile Reporting	6
2.1.3 The Role of Web-Based Review	7
2.2 Related Work and Technical Foundations	7
2.2.1 Informal Reporting Channels	7
2.2.2 Centralized Web Systems	9
2.2.3 API Gateway Pattern	10
2.2.4 Message Queues Between Services	10
2.2.5 Authentication and Role-Based Access	10
2.2.6 Media Evidence and Cloud Storage	11
2.2.7 Database and Cache	11
2.2.8 Shared Infrastructure Package	12
2.2.9 Containerized Deployment	12
2.3 Comparison of Design Approaches	13
3 Methodology	14
3.1 Requirement Analysis	14
3.1.1 Functional Requirements	14
3.1.2 Non-Functional Requirements	15
3.2 Actor Analysis	15
3.2.1 Citizen Actor	15
3.2.2 Relief Actor	16
3.2.3 Administrator Actor	16

3.3	Materials and Development Tools	16
3.4	System Design Method	17
3.4.1	API Gateway Method	17
3.4.2	Message Communication Method	18
3.5	Database Design Method	18
3.5.1	Cache Method	19
3.6	Workflow Design Method	19
3.6.1	Evidence Handling Method	21
3.6.2	Status Review Method	21
3.7	Client Application Method	21
3.7.1	Mobile Application Method	21
3.7.2	Web Dashboard Method	21
3.8	Real-Time Tracking Method	21
3.9	Validation Method	22
4	Implementation and Result	23
4.1	Technique and Tools	23
4.2	Messaging Broker Implementation	25
4.3	Authentication and Authorization Implementation	26
4.3.1	Authentication Mechanism Implementation	26
4.3.2	Refresh Token Implementation	28
4.3.3	Authorization Process Implementation	29
4.4	Report Processing and Evidence Upload Implementation	29
4.4.1	Report Update and Delete Implementation	30
4.5	Public Library Implementation	30
4.6	Real-Time Tracking Implementation	31
4.7	Mobile Citizen Feature Implementation	32
4.7.1	Report List and Profile Access	32
4.7.2	Create Report Implementation	32
4.8	Relief Feature Implementation	33
4.9	Web Dashboard Implementation	34
4.9.1	Evidence Review and Status Update	35
4.10	Deployment Implementation	36
4.11	User Management Implementation	37
5	Evaluation	38
5.1	Evaluation Scope	38
5.2	Functional Evaluation	38
5.3	Backend Evaluation	40
5.4	Client Evaluation	40
5.5	Security and Access Control Evaluation	41
5.6	Deployment and Operations Evaluation	41
5.7	Evaluation Result	42
6	Conclusion and Future Works	43
6.1	Conclusion	43
6.2	Future Works	44
A	API Specifications	47
A.1	Authentication API	47

A.2	Report Creation API	47
B	Interface Screenshots	48
B.1	Mobile Screens	48
B.1.1	Relief and Admin Screens	49
C	Test Cases	51
D	Deployment and Configuration	52
D.1	Deployment Notes	52

List of Tables

3.1	Main materials and tools used in VietFlood development.	17
4.1	Main implementation tools used in VietFlood.	24
4.2	Comparison of message brokers for the VietFlood backend.	25
5.1	Functional evaluation of VietFlood user flows.	39
C.1	Key VietFlood Test Cases	51
D.1	Deployment Configuration Groups	52

List of Figures

2.1	Lifecycle of a VietFlood citizen report from mobile submission to review status.	8
2.2	Comparison between a monolithic backend and the VietFlood microservices structure.	9
2.3	Evidence upload and review flow in VietFlood.	11
2.4	Mobile evidence upload and dashboard evidence review in VietFlood. . . .	12
3.1	Use case diagram for the citizen actor in VietFlood.	15
3.2	Use case diagram for the relief actor in VietFlood.	16
3.3	Use case diagram for the administrator actor in VietFlood.	16
3.4	VietFlood system architecture diagram.	18
3.5	Database schema diagram for VietFlood.	19
3.6	Class or module diagram for the report workflow.	20
3.7	Sequence diagram for creating a VietFlood report.	20
4.1	RabbitMQ communication flow between the API gateway, auth service, reports service, auth queue, and reports queue.	26
4.2	Citizen registration screen in the VietFlood mobile app.	27
4.3	Citizen login screen in the VietFlood mobile app.	27
4.4	Admin login page in the VietFlood web dashboard.	28
4.5	Refresh-token flow from mobile or web client through the API gateway, auth service, refresh-token table, and new access token response.	29
4.6	Report creation flow from mobile form submission to Cloundinary upload, RabbitMQ message, PostgreSQL insert, and Redis cache write.	30
4.7	Shared common library structure showing LoggerService, RedisService, CloundinaryService, and the backend modules that import them.	31
4.8	Socket.IO live tracking flow with send-location, receive-location, location-error, and user-disconnected events.	31
4.9	Citizen report list or citizen home screen showing submitted reports, status labels, and report location information.	32
4.10	Create report form in the VietFlood mobile app.	33
4.11	Evidence upload in the VietFlood mobile app.	33
4.12	Relief report list in the mobile app.	34
4.13	Relief map screen in the mobile app.	34
4.14	Admin dashboard overview with report statistics, search, status filters, report cards, and evidence thumbnails.	35
4.15	Admin evidence review and status update screen for pending, verified, resolved, and rejected reports.	36
4.16	CI/CD workflow from GitHub push to Docker image build, Docker Hub push, Azure App Service configuration, and application restart.	37
4.17	User management view showing citizen, relief, and admin accounts with profile and role information.	37
B.1	Citizen report creation screen.	48
B.2	Evidence upload screen on the mobile app.	48

B.3	Relief worker report list screen.	49
B.4	Admin dashboard overview.	50

List of Abbreviations

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
HTTP	Hypertext Transfer Protocol
UI/UX	User Interface / User Experience
CDN	Content Delivery Network
PUB/SUB	Publish–Subscribe
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
JWT	JSON Web Token
TTL	Time To Live
WS	WebSocket

Abstract

Flood response depends on small pieces of local information becoming useful at the right time. In many cases, the first sign of a flood problem is not an official record but a photo, a short description, or a shared location from a person already near the affected area. When those signals stay inside separate chat groups or informal channels, the information is hard to verify, search, and follow up. This thesis presents VietFlood, a prototype for a Flood Situation Monitoring and Reporting System that organizes citizen flood reports for relief awareness and administrator review.

VietFlood is implemented as a multi-platform prototype. The backend uses NestJS with an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. RabbitMQ supports service-to-service communication, PostgreSQL stores users, refresh tokens, and flood reports through TypeORM, Redis supports cached report lookup, and Cloudinary stores uploaded evidence files. Shared infrastructure code is kept in `vietflood_common`, which provides reusable logging, Redis, and Cloudinary helpers. The mobile application is built with Expo React Native for citizen and relief workflows. The web dashboard is built with Next.js, React, TypeScript, and Tailwind CSS for administrators.

The prototype covers the main workflow from report creation to review. Citizens can register, sign in, create flood reports, provide administrative address fields and coordinates, set urgency and severity, and upload photo evidence. Relief users can open broader report views and map-related screens. Administrators can use the web dashboard to inspect reporter information, search and filter reports, preview evidence, manage users, and update report status using pending, verified, resolved, and rejected states. VietFlood also includes a live tracking path in which connected clients send location updates through Socket.IO after latitude and longitude validation at the gateway.

The work focuses on system design and implementation rather than flood prediction or official emergency dispatch. The final prototype demonstrates how mobile reporting, role-based access control, evidence storage, real-time location events, shared backend utilities, and container-based deployment can be combined in a Vietnam-focused flood reporting system. Although it remains a prototype, VietFlood provides a working foundation for collecting field reports and making them easier to review.

Keywords: VietFlood; flood reporting; relief coordination; real-time tracking; microservice architecture; NestJS; React Native; Next.js; RabbitMQ; PostgreSQL; Redis; Cloudinary.

Chapter 1

Introduction

1.1 The necessity of flood reporting and relief coordination by information technology

1.1.1 What is flood reporting and relief coordination by technology?

Flood information is often noticed first by people who are already in or near the affected area. A resident may see a road becoming unsafe. A volunteer may know that one route is still open while another route is blocked. A nearby user may only have time to send a short description, a photo, or a location. These small reports matter, but they lose value when they are separated from the details needed for review.

In this thesis, technology-supported flood reporting means converting those field observations into structured records that can be stored, searched, and checked later. The idea is related to citizen-generated geographic and crisis information, where local observations can help describe a situation when they are organized for processing and decision support [1, 2]. For VietFlood, a report is treated as more than a message. It should carry the flood category, description, province, ward, address, coordinates, urgency, severity, evidence files, reporter identity, and review status in one record.

VietFlood is developed for this reporting need. It is a multi-platform prototype for a Flood Situation Monitoring and Reporting System in Vietnam. The system brings together a mobile application for citizens and relief users, a web dashboard for administrators, a NestJS backend, and a shared TypeScript infrastructure package [3, 4]. The project does not attempt to forecast floods. Its focus is the later stage, when people have already observed a flood situation and the information needs to be submitted, stored, and reviewed.

The backend project, **VietFlood**, is organized around four main runtime parts: an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. The API gateway is the public entry point for HTTP and WebSocket clients. The authentication service manages registration, sign-in, profile loading, refresh tokens, logout, and user management. The reports service stores report data, evidence metadata, status changes, and cache updates.

The project also includes `vietflood_common`, a shared package used by backend services. It contains common logging, Redis, and Cloudinary helpers. This separation is useful because logging, cache access, and media uploads should not behave differently from one service to another. By keeping this infrastructure code in one package, the backend can reuse the same behavior instead of copying similar helper code into each service.

1.1.2 Some approaches to supporting flood reporting and relief coordination using technology

VietFlood uses several practical approaches to support flood reporting and review. The first is a mobile-first reporting flow. Flood reports are usually created in the field, so the citizen and relief workflows are implemented in an Expo React Native application with TypeScript [5, 6, 4]. The mobile app supports authentication, profile screens, report creation, image picking, location-aware screens, relief views, maps, user overview pages, role-aware navigation, and Vietnamese or English interface text. It also uses the Vietnam Provinces Open API so address fields can be selected more consistently [7].

The second approach is separating responsibilities by role. VietFlood uses citizen, relief, and admin roles. Citizens submit reports and view their own records. Relief users can view broader report information and map-related screens for field awareness. Administrators work from the web dashboard, where they can inspect evidence, search and filter reports, view reporter details, update report status, and manage users. The dashboard checks the current profile before showing admin content, so role control is not treated as only a visual decision in the interface.

The third approach is evidence handling through external media storage. A flood report is easier to review when a photo is attached, but storing large image files directly in PostgreSQL is not suitable for this prototype. VietFlood receives uploaded files through the API gateway, sends them to Cloudinary under the `vietflood/reports` folder, and stores the returned URL, public ID, and resource type with the report data [8]. This allows the dashboard to preview evidence while still keeping enough metadata to delete remote files when a report is removed.

The fourth approach is a small microservice backend. Microservice architecture is commonly described as splitting a system into services around separate capabilities instead of building one large application [9, 10, 11]. In VietFlood, the API gateway communicates with the authentication and reports services through RabbitMQ message patterns [12, 13]. The authentication service consumes messages from `auth_queue`, while the reports service consumes messages from `reports_queue`. PostgreSQL stores persistent data through TypeORM, and Redis is used for cached report lookup data [14, 15, 16]. This design is more complex than a single server, but it makes the responsibility of each service easier to explain and maintain.

The fifth approach is live location broadcasting. VietFlood includes a Socket.IO tracking path where connected clients can send location updates [17]. The gateway validates latitude and longitude before broadcasting the update to other clients. Invalid payloads return an error event. This is not a complete emergency dispatch feature, but it demonstrates how live movement information can be added to the reporting system.

1.2 Problem Statement

Flood reports are less useful when the important pieces are scattered. A photo without a location may not tell an administrator where to look. A coordinate without a description may not explain the risk. A report without a status makes it hard to know whether someone has reviewed it. These problems become more serious when many reports arrive during heavy rain.

The main problem addressed in this thesis is:

How can VietFlood collect flood reports from citizens, store the report data

and evidence safely, and present the reports in a form that relief users and administrators can review?

This problem includes several design concerns. The report form must collect enough information to be useful while still being quick enough for a mobile user. Evidence files must remain connected to the report record after upload. Report status must be visible so that pending, verified, resolved, and rejected cases can be separated. The backend must support both the mobile application and the web dashboard without forcing each client to duplicate the same business rules.

The thesis also defines what VietFlood does not solve. It does not provide flood prediction, rescue routing, satellite image processing, SMS warning delivery, official dispatch integration, offline-first synchronization, or formal GIS analysis. Those features would require different data sources, operational partners, and safety validation. The scope here is the prototype that was designed and implemented: account handling, report submission, evidence upload, role-based review, live location events, caching, and container-based deployment.

1.3 Scope And Objectives

The scope of VietFlood is divided into four project modules. The first module is the backend service group, **VietFlood**. It contains the API gateway, authentication service, reports service, and tracking gateway. The gateway exposes authentication and report routes, receives evidence uploads, and handles Socket.IO events. The authentication service owns user accounts and tokens. The reports service owns report records, evidence metadata, status updates, and report cache invalidation.

The second module is **vietflood_common**. This shared package provides backend infrastructure that is reused across services. It includes structured logging, Redis access, and Cloudinary upload or deletion helpers. The logger can include service name, trace context, user ID, role, request path, and method when request context is available.

The third module is **VietFlood_mobile**. It is the Expo React Native application for citizen and relief workflows. It includes login, registration, profile screens, report creation, image attachment, citizen report lists, relief report details, map screens, user overview screens, settings, and localization support. The app also normalizes backend response data because report coordinates and field names may appear in different forms such as **lat**, **lng**, latitude, longitude, snake case, and camel case.

The fourth module is **Vietflood_web**. It is a Next.js dashboard for administrator monitoring. The dashboard signs in through the backend, stores access and refresh tokens, refreshes sessions when required, loads the current profile, blocks non-admin accounts from dashboard use, and displays reports with search, status filters, evidence previews, reporter information, and status update actions.

The deployment scope covers Docker Compose for the backend services, Redis, and RabbitMQ [18]. The production-oriented setup uses Docker Hub images and Azure App Service [19]. GitHub Actions builds and tags service images, pushes them to Docker Hub, configures the Azure App Service deployment, and restarts the application [20]. Environment variables are used for database URLs, RabbitMQ credentials, Redis password, JWT secrets, refresh-token secrets, Cloudinary credentials, and admin seed values.

1.3.1 Goals of VietFlood

The main goal of VietFlood is to deliver a working flood reporting prototype for Vietnam. The prototype should allow citizens to submit reports from mobile devices, preserve photo evidence, enforce role-based access, and give administrators a readable interface for report follow-up.

The specific goals are:

1. Build a NestJS backend that separates public access, authentication, report handling, and live tracking.
2. Use RabbitMQ message patterns for communication between the API gateway and internal services.
3. Store user accounts, refresh tokens, and flood reports in PostgreSQL through TypeORM.
4. Protect citizen, relief, and admin workflows with JWT access tokens, refresh tokens, and role guards.
5. Support report creation with category, description, address fields, coordinates, urgency, severity, and photo evidence.
6. Upload evidence to Cloudinary and keep the returned URL and public ID with the report record.
7. Use Redis for report lookup caching and clear stale cached data after report changes.
8. Provide an Expo React Native mobile app for citizen reporting and relief-oriented views.
9. Provide a Next.js web dashboard for admin login, report search, evidence review, user management, and status updates.
10. Prepare Docker Compose and CI/CD configuration for local and production-style backend deployment.

1.3.2 System Actors and Basic Functions

VietFlood uses three roles: citizen, relief, and admin. The role model is intentionally simple because the prototype only needs enough separation to protect personal reports, field awareness screens, and administrator review actions.

Citizens use the mobile app to register, sign in, manage their profile, create flood reports, attach evidence, provide location information, and view reports linked to their own account. This actor represents people who are close enough to the flood situation to submit first-hand information.

Relief users use the mobile app for wider awareness. They can inspect report lists, open report details, and use map-related views. In this prototype, relief users do not verify or reject reports. Their role is focused on reading and field awareness rather than system administration.

Administrators use the web dashboard. They can sign in, load reports, search by report or reporter information, filter by status, preview evidence, inspect reporter details,

update report status, and manage user records. The dashboard checks the signed-in profile before allowing access to admin pages.

All three roles connect through the same backend entry point. Mobile and web clients call the API gateway, and the gateway decides whether the request should go to authentication, report handling, evidence upload, or live tracking. This keeps client applications focused on user interaction while backend services handle system rules.

1.3.3 Thesis Structure

This thesis is organized into six chapters. Chapter 1 introduces the motivation, problem statement, scope, objectives, actors, and overall structure of VietFlood. Chapter 2 reviews background and related work on flood reporting, citizen-generated information, role-based review, microservices, message queues, media evidence, caching, real-time communication, and deployment.

Chapter 3 presents the methodology, including the project modules, actors, architecture, database design, use cases, workflows, deployment method, and evaluation plan. Chapter 4 describes the implementation and result of the backend, mobile application, web dashboard, shared library, and CI/CD setup. Chapter 5 evaluates the prototype and discusses current limitations. Chapter 6 concludes the thesis and outlines future work.

Chapter 2

Background and Related Works

2.1 Background

Flood reporting is a practical information problem before it becomes a software problem. A person near a flooded road may know what is happening before any official record is created, but that first message is often incomplete. It may include a photo but no ward name, a description but no coordinates, or a location pin without enough context to judge the situation. VietFlood is built around this gap. The system tries to turn quick local observations into structured reports that can be stored, searched, and reviewed later. This direction is consistent with the use of volunteered and citizen-generated information in crisis response, where local observations become more useful when they are organized for processing [1, 2].

The project does not aim to replace forecasting systems, hydrological models, or official emergency command centers. Its role is narrower and more concrete. VietFlood supports report submission from a mobile device, evidence upload, report storage, status tracking, relief awareness screens, and administrator review through a web dashboard. This scope is important because a flood management platform can become very broad. The thesis focuses on the part that was designed and implemented: moving field information from a citizen report into a backend record that an authorized user can inspect.

2.1.1 Flood Reports as Structured Field Records

An informal flood message can be fast, but it is not always easy to reuse. A chat message may be buried after a few minutes. A photo may be forwarded without its original location. A phone call may help one person but leave no shared record for others. VietFlood treats each citizen submission as a structured field record instead of a loose message.

The report entity in VietFlood stores the category, description, image links, evidence metadata, province, ward, address line, latitude, longitude, urgency flag, severity value, status, timestamps, and the account that created the report. These fields answer the questions that usually come first during review: what happened, where it happened, who reported it, how serious it appears, and what evidence supports the report.

The report status model is intentionally small. A report can be **pending**, **verified**, **resolved**, or **rejected**. A new submission begins as pending. An administrator can mark it as verified when the information is considered reliable, resolved when the case is no longer active, or rejected when the report is unsuitable or incorrect. Keeping rejected reports separate is useful because it preserves an audit trail instead of simply deleting every bad report.

2.1.2 The Role of Mobile Reporting

Mobile reporting is central to VietFlood because most observations are made away from a desk. A citizen may be standing near a flooded street, using a phone connection, and trying to submit a report quickly. For that reason, the citizen and relief-side application is built with Expo React Native [5, 6]. The same mobile codebase supports authentication,

profile pages, report creation, image selection, report lists, relief-oriented screens, maps, and role-aware navigation.

Figure 2.1 presents the normal life of a report in the prototype. A citizen creates the report on the mobile app. The request reaches the API gateway, evidence files are uploaded, the reports service stores the record, and an administrator can later inspect the case and update its status. The diagram is kept simple because Chapter 2 explains the background idea. The detailed service architecture is discussed later in the methodology chapter.

The mobile form also supports location consistency. VietFlood uses the Vietnam Provinces Open API so users can select province, district, and ward information from known administrative divisions instead of typing every field freely [7]. This reduces spelling variation and makes the data easier to filter. Coordinates are also part of the report model, and the mobile code handles different field names such as `lat`, `lng`, `latitude`, and `longitude` when normalizing backend responses.

2.1.3 The Role of Web-Based Review

The web application has a different purpose from the mobile app. It is not designed for quick field submission. It is designed for administrator review, where a user may need to scan many reports, inspect evidence, search by report or reporter information, and update status. VietFlood’s web dashboard is built with Next.js, React, TypeScript, and Tailwind CSS [21, 22, 4, 23].

The dashboard receives report data through the same backend gateway used by the mobile app. It displays report status, category, severity, address, description, evidence thumbnails, reporter information, and creation time. Search and filtering are important because a report list becomes difficult to use when many cases are pending at the same time. Administrators can focus on a status group, a reporter, or a location-related keyword instead of manually reading every card.

Access control is part of the review design. A citizen account should not enter the administrator dashboard simply by opening a page URL. The web client checks the signed-in profile before showing admin pages, and backend routes use JWT and role-based guards where protected access is needed. This keeps interface behavior and backend permission checks aligned.

2.2 Related Work and Technical Foundations

The technical foundation of VietFlood comes from several common patterns rather than one copied application. The project combines mobile field reporting, role-based access, a small microservice backend, message queues, external media storage, caching, real-time socket communication, and containerized deployment. Each part solves a specific problem in the reporting workflow.

2.2.1 Informal Reporting Channels

In real communities, flood information often begins in informal channels such as phone calls, messaging apps, social media posts, or volunteer groups. These channels are fast because people already know how to use them. A resident can send a photo in seconds. A volunteer can forward a warning to a group. For early awareness, that speed is useful.

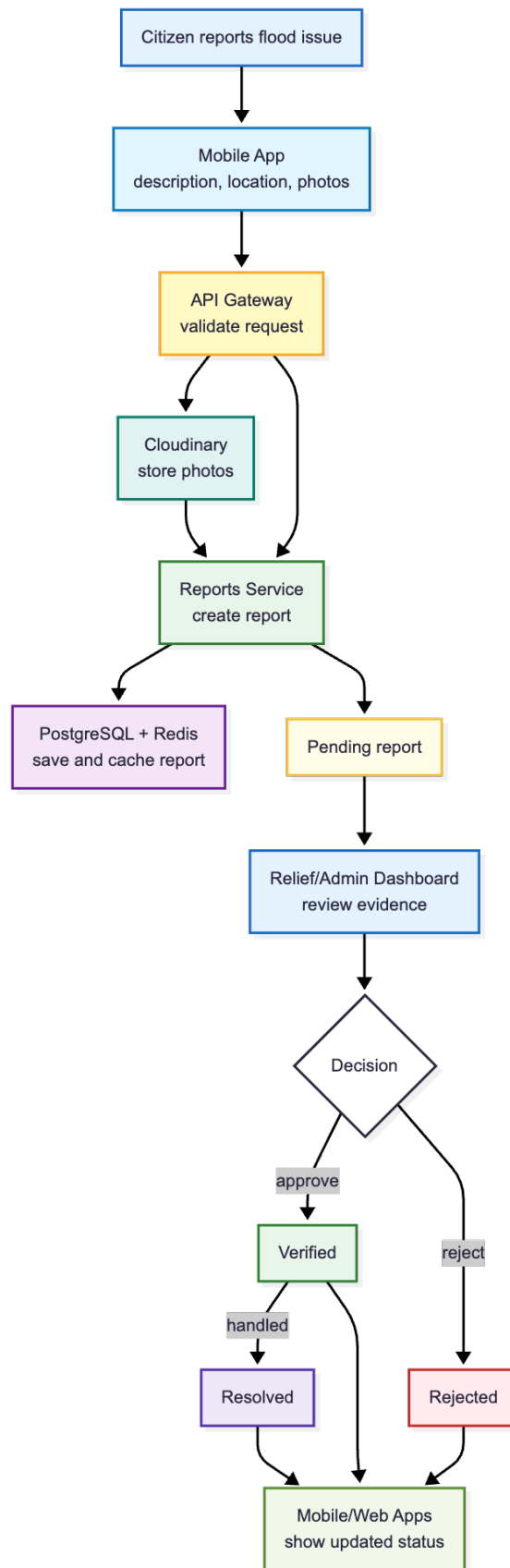


Figure 2.1: Lifecycle of a VietFlood citizen report from mobile submission to review status.

The weakness appears after the first wave of messages. Informal channels do not naturally create a clean report database. Duplicate reports, missing addresses, old photos, unclear status, and inconsistent reporter information can all appear in the same conversation. It is also hard to know whether a message has been reviewed, verified, resolved, or rejected.

VietFlood keeps the citizen reporting idea but adds structure. The mobile app asks for report details, location information, urgency, severity, and evidence. The backend stores the submission as a report record, and the dashboard gives administrators a shared place to search and review it. The system therefore does not replace human judgment; it gives that judgment a clearer record to work with.

2.2.2 Centralized Web Systems

One simple way to build a reporting system is to use a centralized web application. A single backend can handle login, users, reports, files, status updates, and admin pages. This approach is easy to understand and can be suitable for a small application. It also reduces the number of moving parts during development.

VietFlood uses a small microservice design instead. This choice was made because the project needs clear boundaries between public access, authentication, report handling, media upload, caching, and live tracking. Microservice architecture is often described as a way to separate a system around business or technical capabilities, while accepting extra operational cost [9, 11, 10, 24]. VietFlood keeps the split modest: an API gateway, an authentication service, a reports service, Redis, RabbitMQ, PostgreSQL, and Cloudinary integration.

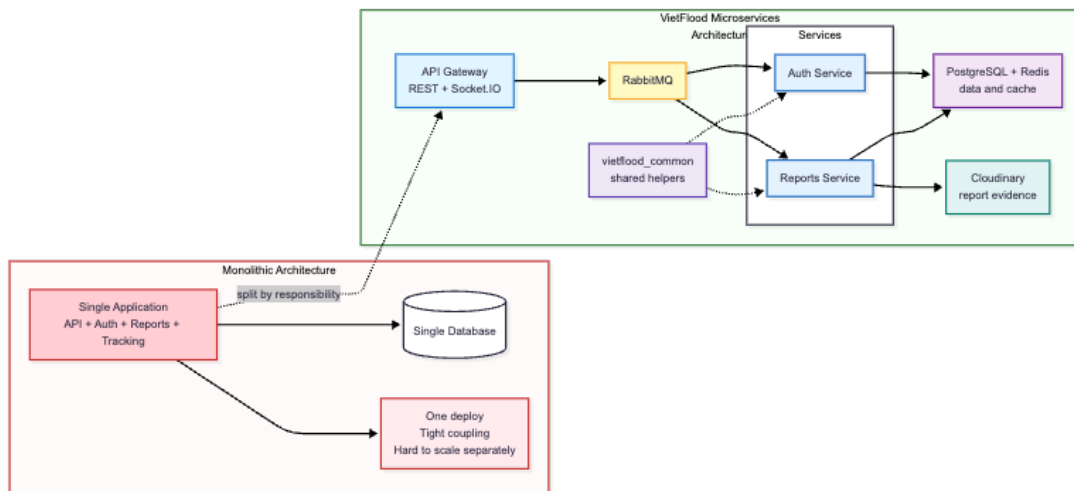


Figure 2.2: Comparison between a monolithic backend and the VietFlood microservices structure.

Figure 2.2 shows the difference between a single backend and the VietFlood structure. In VietFlood, the API gateway handles the public interface, the authentication service owns identity and tokens, and the reports service owns flood reports and evidence metadata. This division is not meant to make the prototype larger for its own sake. It is meant to keep the responsibilities visible.

2.2.3 API Gateway Pattern

The API gateway is the public entrance to the VietFlood backend. Mobile and web clients do not call the authentication service or reports service directly. They call the gateway, and the gateway forwards the request to the correct internal service. This follows the common gateway pattern, where a client-facing component hides internal service structure and provides one stable API surface [24, 3].

In VietFlood, the gateway handles authentication routes, report routes, multipart evidence upload, and Socket.IO tracking events. For report creation, it receives uploaded files through the `files` field, sends the file buffers to Cloudinary, and forwards the report payload and evidence metadata to the reports service. This keeps file upload concerns close to the public HTTP layer while keeping report persistence inside the reports service.

The gateway also hosts the live tracking entry point. A connected client can send a `send-location` event. The gateway checks latitude and longitude before broadcasting a `receive-location` event. Invalid payloads return `location-error`, and disconnected sockets are announced through `user-disconnected`. These events are small, but they show how real-time data can be added without changing the normal report API.

2.2.4 Message Queues Between Services

RabbitMQ is used for communication between the API gateway and internal backend services [12, 13]. The gateway sends authentication-related messages to the auth queue and report-related messages to the reports queue. The corresponding service consumes the message, performs the work, and returns the result.

This design makes service boundaries clearer. For example, the gateway receives a sign-in request, but password checking and token creation belong to the authentication service. The gateway receives a create-report request, but report persistence belongs to the reports service. The gateway coordinates the request; it does not own all business logic.

Message-based communication also changes how errors are handled. A service may be unavailable or slow, so the gateway needs controlled timeouts and predictable responses. This is more work than a direct function call, but it reflects the realities of distributed systems and gives the project a useful technical foundation.

2.2.5 Authentication and Role-Based Access

VietFlood uses password-based authentication, JWT access tokens, refresh tokens, and role checks. Passwords are hashed with bcrypt before storage [25]. After sign-in, the system returns an access token and refresh token. The access token follows the JWT model, where signed claims can be checked by protected routes [26].

The project uses three roles: citizen, relief, and admin. Citizens create reports and view reports linked to their own account. Relief users use mobile screens for broader report awareness and map-related viewing. Administrators use the web dashboard to inspect reports, update status, and manage users. This separation matters because report review and user management should not be available to every signed-in user.

The mobile application uses role-aware navigation so users see screens that match their role. The web dashboard checks the current profile before allowing admin pages to appear. The backend protects sensitive operations with authentication and role guards. Together, these layers keep access control from depending only on the user interface.

2.2.6 Media Evidence and Cloud Storage

Photo evidence can make a flood report easier to judge. A description can say that a street is flooded, but an image can show water level, road blockage, damage, or surrounding conditions. VietFlood stores uploaded evidence in Cloudinary instead of storing raw image files directly in PostgreSQL [8].

The upload path begins at the API gateway. The gateway receives file buffers from the mobile app, uploads them to the `vietflood/reports` folder in Cloudinary, and passes the returned URL, public ID, and resource type to the reports service. The reports service stores this evidence metadata with the report. The URL supports preview and display. The public ID supports cleanup when a report is removed.

This design separates media storage from report storage. PostgreSQL keeps structured report data, while Cloudinary handles media files and delivery. Shared Cloudinary helpers in `vietflood_common` keep upload and delete behavior reusable across services.

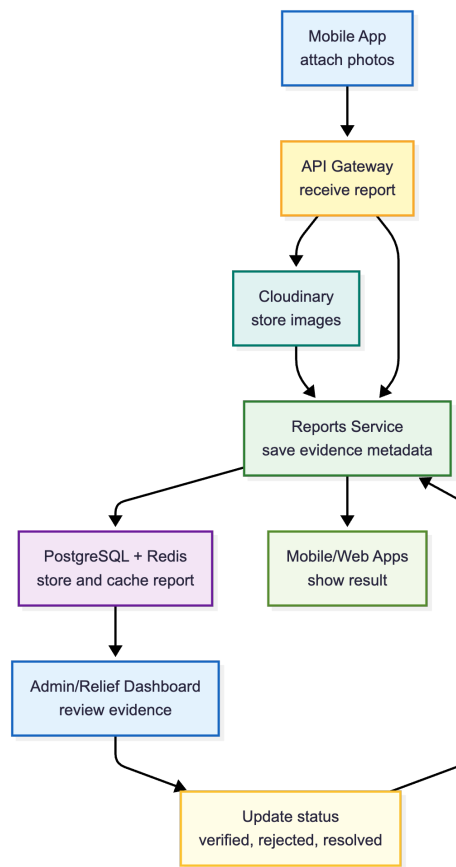


Figure 2.3: Evidence upload and review flow in VietFlood.

Figure 2.3 shows the technical path of evidence upload. The next figure pairs that system flow with screenshots from the actual application so the reader can connect the architecture to the user experience.

2.2.7 Database and Cache

PostgreSQL is the main persistent database for VietFlood. The backend uses TypeORM to map application entities to database tables [14]. Reports store both ordinary relational fields, such as status and coordinates, and JSON evidence metadata. PostgreSQL supports JSON data when flexible structured fields are needed inside a relational database

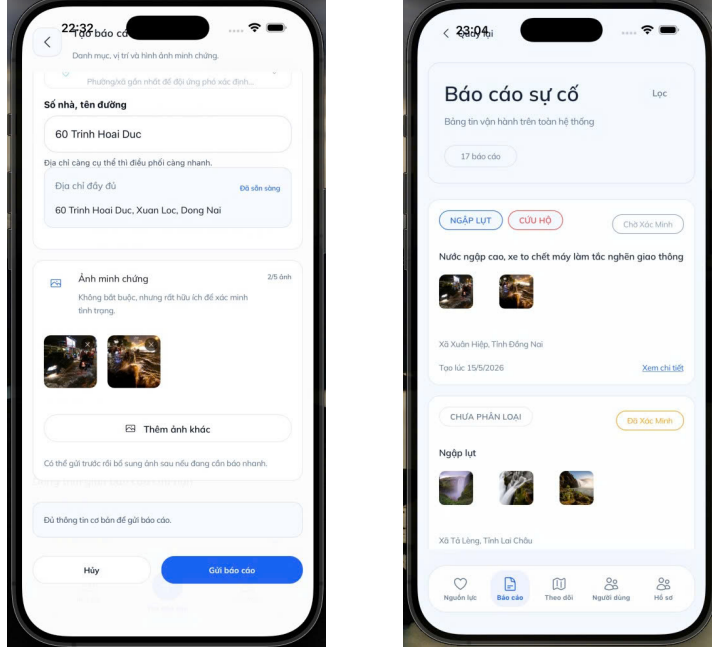


Figure 2.4: Mobile evidence upload and dashboard evidence review in VietFlood.

[15].

Redis is used as a focused cache rather than a replacement for the database [16]. The reports service can cache report lookup data and clear stale entries when a report changes. PostgreSQL remains the source of truth, while Redis helps reduce repeated reads for data that may be requested multiple times.

The cache scope is deliberately limited. A broad cache layer can become difficult to reason about if invalidation is unclear. VietFlood uses caching where the key pattern and update behavior are understandable, which fits the size of the prototype.

2.2.8 Shared Infrastructure Package

The backend services reuse common infrastructure through the `vietflood_common` package. This package exports the Redis module and service, Cloudinary module and service, and a structured logger. The logger can include timestamp, log level, service name, trace context, user ID, role, request path, and HTTP method when that context is available.

The shared package reduces repeated code across the gateway, authentication service, and reports service. More importantly, it keeps infrastructure behavior consistent. A Redis connection should be configured the same way across services. A Cloudinary upload should return evidence metadata in the same shape. A log message should carry enough context to trace work through the backend.

2.2.9 Containerized Deployment

VietFlood uses Docker Compose to run the backend services together with Redis and RabbitMQ [18]. The local setup builds service images from the repository, while the production-oriented configuration uses Docker Hub images for the gateway, auth service, and reports service. Environment variables provide database URLs, Redis settings, RabbitMQ credentials, JWT secrets, refresh-token secrets, Cloudinary credentials, and admin seed values.

The project also includes a GitHub Actions deployment workflow. The workflow builds

service images, tags them, pushes them to Docker Hub, configures Azure App Service, and restarts the deployed application [20, 19]. This is aligned with continuous delivery practices, where builds and deployments should be repeatable instead of depending on manual steps [27, 28].

2.3 Comparison of Design Approaches

VietFlood can be compared with two simpler approaches. The first is informal reporting through chat, phone calls, and social media. This is fast and familiar, but it does not produce a searchable report database or a clear status history. The second is a single-server reporting website. This is easier to develop, but authentication, reports, evidence, caching, and live tracking can become tightly connected as the project grows.

The VietFlood approach sits between those two options. It is more structured than informal reporting and more modular than a single backend, but it is still small enough to explain clearly. The API gateway owns the public interface. The authentication service owns accounts and tokens. The reports service owns flood report data and evidence metadata. Redis, PostgreSQL, RabbitMQ, Cloudinary, and Socket.IO each support a specific infrastructure need.

This design adds engineering effort. There are more services to configure, more environment variables to manage, and more failure points to consider. The benefit is that each responsibility is visible. For a thesis prototype, that tradeoff is useful because it demonstrates not only a working application, but also the architectural reasoning behind the application.

Chapter 3

Methodology

This chapter describes the method used to design and implement VietFlood. The work followed a software-engineering approach rather than an experimental survey method. First, the main actors and reporting workflow were defined. Second, the report data model was shaped around location, evidence, owner, urgency, severity, and review status. Third, the system was divided into backend services, shared infrastructure utilities, a mobile application, and a web dashboard. Finally, the main workflows were checked through source-level review, local deployment, and end-to-end use of the reporting path. This follows the architectural view that system structure should be guided by responsibilities and quality needs, not only by the chosen technology stack [24].

3.1 Requirement Analysis

The requirement analysis started from the central workflow of the project: a person observes a flood-related situation, submits a report from a mobile device, and an authorized reviewer later inspects the report. VietFlood does not attempt to solve the whole disaster-response process. The method focuses on the part that can be implemented and tested in the prototype: account handling, report submission, evidence upload, report review, and status tracking.

3.1.1 Functional Requirements

The functional requirements were grouped by the three roles supported by the system.

- **Citizen users** can register, sign in, update their profile, create flood reports, provide address and coordinate information, attach photo evidence, and view reports linked to their own account.
- **Relief users** can view broader report lists, open report details, inspect map-related screens, and use report information for field awareness. They do not verify or reject reports in this prototype.
- **Admin users** can use the web dashboard to view all reports, search and filter records, inspect reporter information, preview evidence, update report status, and manage user information.

The report object is the center of the system. Each report needs category data, a written description, administrative address fields, coordinates, evidence metadata, a reporter, a severity value, an urgency flag, timestamps, and a review status. The status values are kept simple: **pending**, **verified**, **resolved**, and **rejected**. This small set is enough for the prototype and avoids a review process that would be too complex for the current scope.

3.1.2 Non-Functional Requirements

The non-functional requirements came from the way the system is expected to be used. The backend should expose one public entry point for mobile and web clients. Authentication and report handling should be separated so identity logic does not become mixed with report storage and evidence handling. Uploaded images should be stored outside the relational database, while report records and evidence metadata remain searchable. The backend should also be deployable through repeatable configuration rather than manual server setup.

These requirements led to four main design decisions. VietFlood uses an API gateway in front of internal services. RabbitMQ carries messages between the gateway, authentication service, and reports service. Cloudinary stores uploaded evidence files, while PostgreSQL stores report records and evidence metadata. Docker Compose is used to run the backend services together with Redis and RabbitMQ in a repeatable environment.

3.2 Actor Analysis

VietFlood has three main actors: citizen, relief, and administrator. The actors were chosen to match the real responsibilities in the prototype. Citizens submit reports, relief users read field information, and administrators review and manage the submitted records.

3.2.1 Citizen Actor

The citizen actor represents a person who notices a flood-related incident and submits the first structured record. The citizen workflow begins with registration or sign-in. After authentication, the user can create a report by selecting a category, writing a description, choosing location information, setting urgency and severity, and attaching optional photos. The mobile app can also help with device location and Vietnam administrative division selection. The mobile app can also help with device location and Vietnam administrative division selection.

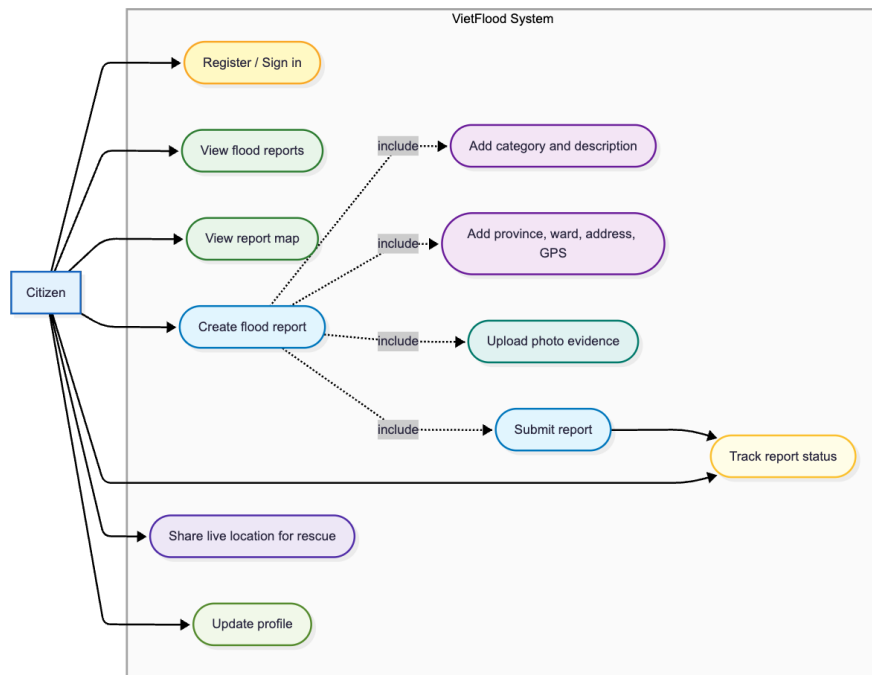


Figure 3.1: Use case diagram for the citizen actor in VietFlood.

3.2.2 Relief Actor

The relief actor represents users who need situational awareness from the submitted reports. In the mobile app, this role can use report lists, report detail screens, and map-related views. The relief role is deliberately narrower than the admin role. Relief users can inspect information for field awareness, but they do not verify, reject, or administratively close reports in this thesis version.

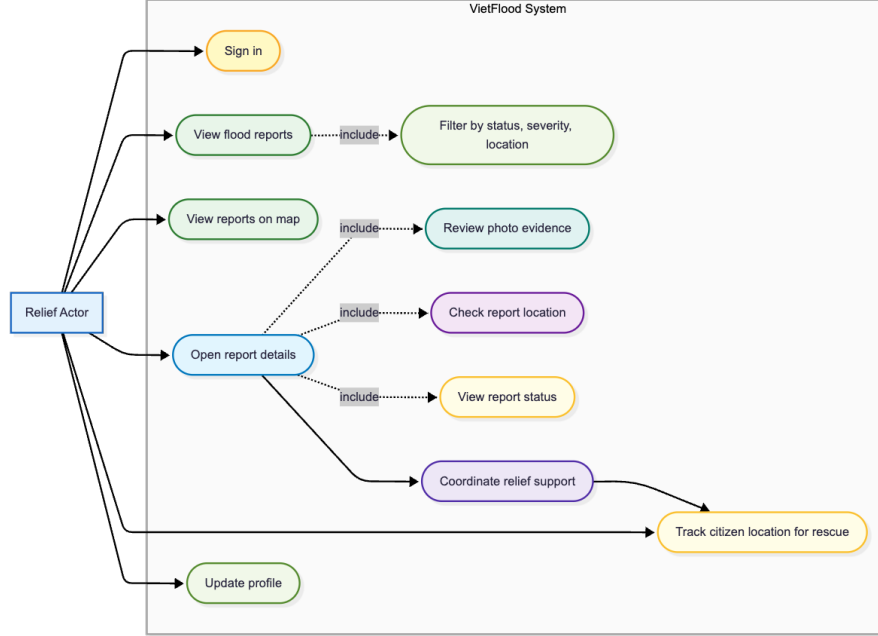


Figure 3.2: Use case diagram for the relief actor in VietFlood.

3.2.3 Administrator Actor

The administrator actor works through the web dashboard. The dashboard loads report data through the API gateway and displays report content together with reporter details, status, evidence previews, and filtering controls. Admin users are responsible for changing report status and managing user records. The web client checks the signed-in profile before showing the dashboard so a non-admin account cannot simply open the admin pages.

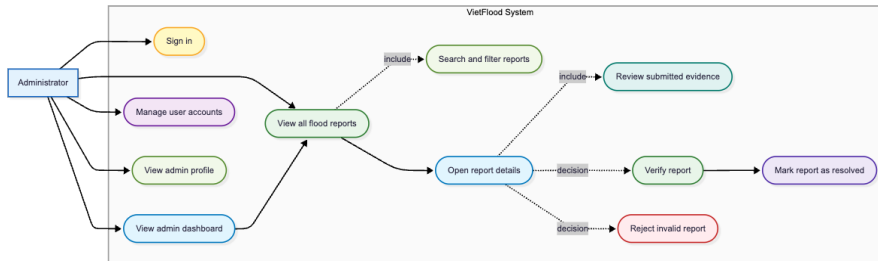


Figure 3.3: Use case diagram for the administrator actor in VietFlood.

3.3 Materials and Development Tools

VietFlood was developed from four source-code modules. The backend module contains the NestJS API gateway and microservices [3, 12]. The shared module contains reusable

logging, Redis, and Cloudinary utilities. The mobile module contains the Expo React Native application [5, 6]. The web module contains the Next.js administrator dashboard [21]. The main tools are summarized in Table 3.1.

Table 3.1: Main materials and tools used in VietFlood development.

Area	Tools and purpose
Backend	NestJS, TypeScript, TypeORM, PostgreSQL, RabbitMQ, Redis, Socket.IO, JWT, bcrypt.
Shared package	<code>vietflood_common</code> for structured logging, Redis client setup, and Cloudinary file operations.
Mobile client	Expo, React Native, TypeScript, React Navigation, Redux Toolkit, image picking, secure token storage, device location, and Vietnam Provinces Open API.
Web dashboard	Next.js, React, TypeScript, Tailwind CSS, protected admin login, token refresh, report overview, search, filtering, evidence preview, and status update.
Runtime infrastructure	Docker, Docker Compose, PostgreSQL, Redis, RabbitMQ, Cloudinary, and Azure App Service configuration.

3.4 System Design Method

The backend was designed as a small microservice system. The service split was kept limited because too many services would make the prototype harder to operate without adding much value. Microservice literature emphasizes that service boundaries should clarify responsibility and support change, while still accounting for operational cost [9, 11, 10]. VietFlood therefore uses only the boundaries needed for the thesis: public access, authentication, report management, shared infrastructure utilities, and supporting infrastructure.

The API gateway receives HTTP and WebSocket traffic from clients. The authentication service owns registration, sign-in, refresh tokens, profile loading, logout, admin seeding, and user management. The reports service owns report creation, report lookup, evidence metadata, status changes, cache updates, and report deletion. Redis supports cached report lookup, RabbitMQ carries service messages, PostgreSQL stores persistent data, and Cloudinary stores uploaded images.

3.4.1 API Gateway Method

The gateway exposes the routes used by both client applications. Authentication routes are grouped under `/auth`, and report routes are grouped under `/reports`. Report creation uses multipart upload through the `files` field. The gateway holds uploaded files in memory long enough to upload them to Cloudinary. It then forwards the report data and evidence metadata to the reports service.

The gateway also applies authentication and role checks. Citizen routes are used for personal report operations. Broader report-reading routes are available to relief and admin

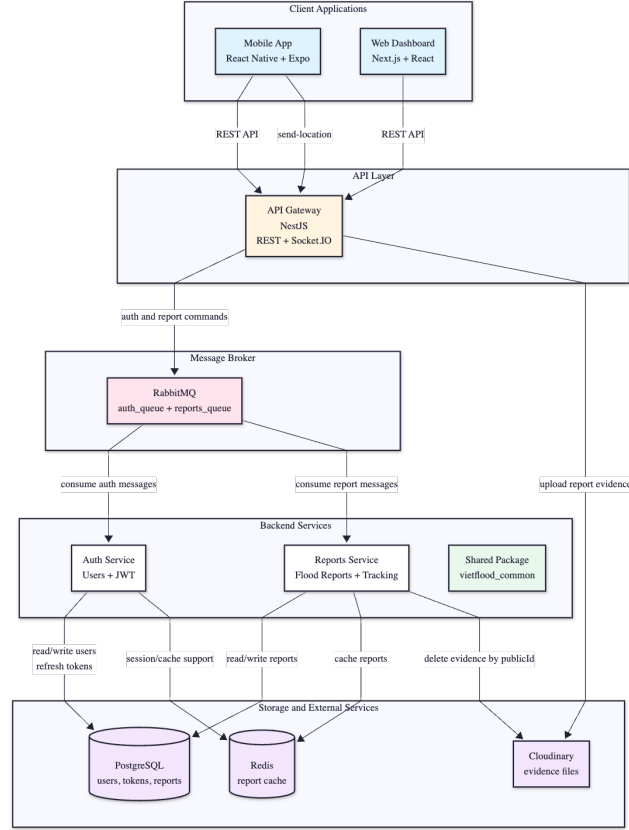


Figure 3.4: VietFlood system architecture diagram.

users. Admin-specific operations are protected separately. This method keeps the clients simple while still enforcing access rules on the backend.

3.4.2 Message Communication Method

RabbitMQ is used for communication between the gateway and internal services [13]. Authentication messages are consumed by the auth service, while report messages are consumed by the reports service. The authentication service handles message patterns for registration, sign-in, profile loading, token refresh, logout, and user operations. The reports service handles report creation, listing, lookup, update, and deletion.

This message-based method makes boundaries visible during development. A failed report submission can be inspected at the gateway request, the Cloudinary upload step, the RabbitMQ message, or the reports service handler. The gateway also uses controlled service-call behavior so a slow internal service does not leave the client waiting without a response.

3.5 Database Design Method

The database design was derived from the report workflow. A report needs a user owner, location fields, status, urgency, severity, evidence metadata, and timestamps. The backend uses PostgreSQL as the main database and TypeORM entities to map application objects to database tables [14, 15].

The user entity stores email, username, phone, hashed password, role, name fields, date of birth, address line, ward, province, and timestamps. Email, username, and phone

are unique. The supported roles are **citizen**, **relief**, and **admin**.

The report entity stores category arrays, description, image URLs, evidence JSON, province, ward, address line, latitude, longitude, status, reporter name, urgency, severity, timestamps, and owner user ID. Evidence is stored as JSON metadata because each uploaded item needs a URL, Cloudinary public ID, and resource type. PostgreSQL supports this kind of flexible structured field through JSON storage [15].

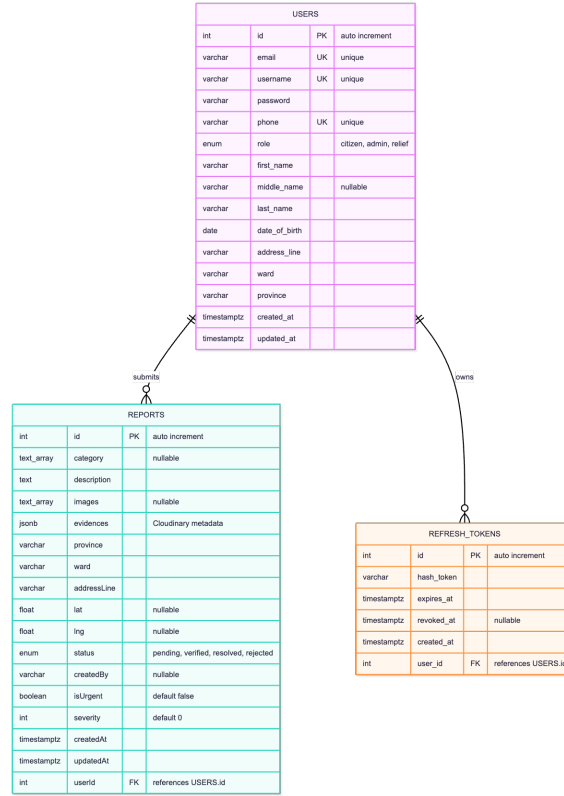


Figure 3.5: Database schema diagram for VietFlood.

3.5.1 Cache Method

Redis is used as a cache and not as the source of truth [16]. When a report is created, the reports service stores a cached copy using a key pattern based on the report ID. When a report is requested, the service can check Redis before reading from PostgreSQL. When the report is updated or deleted, the cache entry is refreshed or removed.

The cache scope is intentionally narrow. Report ownership, status, evidence, and location remain in PostgreSQL. Redis only provides a temporary lookup copy. This keeps invalidation understandable for the prototype.

3.6 Workflow Design Method

The main workflow is citizen report creation. The citizen fills in category, description, administrative location, coordinates, urgency, severity, and optional image evidence. The mobile app sends the report as a multipart request. The API gateway authenticates the user, uploads evidence files to Cloudinary, and forwards the completed payload to the reports service through RabbitMQ. The reports service saves the report in PostgreSQL and creates or updates the Redis cache entry.

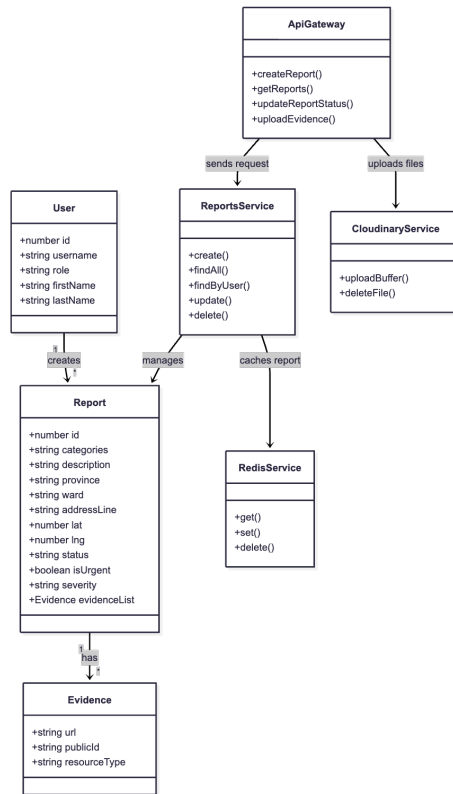


Figure 3.6: Class or module diagram for the report workflow.

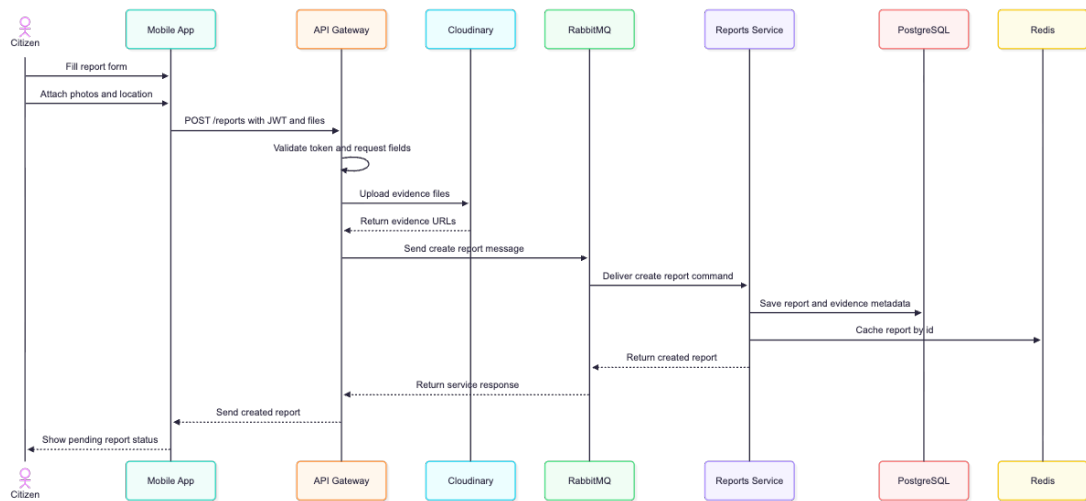


Figure 3.7: Sequence diagram for creating a VietFlood report.

3.6.1 Evidence Handling Method

Evidence images are not stored in PostgreSQL as raw files. The gateway uploads image buffers to Cloudinary under the report folder [8]. The reports service stores the returned URL, public ID, and resource type with the report. The URL is used by the mobile app and dashboard for display. The public ID is kept so related media can be removed when a report is deleted.

This method keeps the database focused on structured data while Cloudinary handles media storage and delivery. It also means media credentials must be provided through environment configuration instead of being written into the source code.

3.6.2 Status Review Method

Status review is handled as an administrator workflow. Relief users can read broader report information and use map-related screens, but verification and rejection are not part of the relief role in this thesis. The administrator dashboard sends status updates through the gateway. The reports service checks the existing report, merges new evidence metadata when needed, updates the status, and refreshes the cache entry.

3.7 Client Application Method

3.7.1 Mobile Application Method

The mobile app was developed with Expo React Native and TypeScript [5, 6, 4]. It supports token-based authentication, profile screens, report creation, report lists, relief report details, map screens, settings, localization, and role-aware navigation. Redux Toolkit supports application state around authentication and client-side data flow [29].

The mobile report service normalizes backend data before rendering it. This is needed because report fields may appear in different naming styles, such as snake case and camel case, or as `lat/lng` and `latitude/longitude`. The mobile location method also combines administrative division data from the Vietnam Provinces Open API with device location when permission is available [7].

3.7.2 Web Dashboard Method

The web dashboard was developed with Next.js, React, TypeScript, and Tailwind CSS [21, 22, 4, 23]. It is focused on administrator review. The dashboard signs in through the backend, stores access and refresh tokens, refreshes sessions when needed, loads the current profile, and blocks non-admin users from the admin workspace.

After authentication, the dashboard loads report data from the gateway and displays report cards with status, category, severity, address, description, reporter information, and evidence thumbnails. It also provides search, filtering, user management, and status update controls.

3.8 Real-Time Tracking Method

The backend includes a Socket.IO gateway for live location updates [17]. A connected client can emit a location event with latitude, longitude, accuracy, heading, speed, and timestamp. The gateway checks latitude and longitude ranges before broadcasting the

accepted location to other clients. Invalid payloads receive an error event. When a socket disconnects, the gateway announces the disconnected client.

This feature is treated as a prototype path. It proves that the backend can receive and broadcast live location data. Future work would still be needed for authenticated socket sessions, route assignment, persistent location history, and operational use in real relief coordination.

3.9 Validation Method

Validation was performed through source-level checks and workflow checks. Source-level checks focused on whether each module matched its responsibility: the gateway handled client-facing access, the authentication service handled identity and tokens, the reports service handled report data, and the shared package handled reusable infrastructure. Docker Compose was used as the repeatable local environment for backend services and infrastructure dependencies [18].

Workflow checks focused on the main paths of the prototype:

- citizen registration, sign-in, and profile loading;
- citizen report creation with category, address, coordinates, urgency, severity, and images;
- evidence upload to Cloudinary and metadata storage in PostgreSQL;
- report list loading for citizen, relief, and admin views;
- administrator report status update through the web dashboard;
- Redis cache creation, refresh, and deletion during report changes;
- Docker Compose startup for backend services, Redis, and RabbitMQ.

This validation method matches the current stage of VietFlood. The system is a working prototype, not a certified emergency-response platform. The checks therefore focus on whether the main reporting, evidence, review, cache, and service-integration flows operate according to the design.

Chapter 4

Implementation and Result

This chapter explains how the VietFlood prototype was built after the design decisions in the previous chapter. The discussion follows the same implementation order as the report format: development tools, service communication, authentication, report processing, shared backend utilities, real-time tracking, mobile features, web dashboard features, deployment, and user management.

The implementation is divided across four repositories: the NestJS backend, the mobile application, the web dashboard, and the shared common package. The backend provides the API gateway, authentication service, reports service, message queues, cache access, media handling, and live tracking gateway. The mobile application supports citizen reporting and relief viewing. The web dashboard supports administrator review and report status control. The shared package keeps repeated infrastructure code in one place so the services can stay focused on their own responsibilities.

4.1 Technique and Tools

VietFlood was implemented as a multi-platform system rather than as a single application. The backend was written with NestJS and TypeScript [3, 4]. The mobile client was built with Expo React Native [5, 6]. The administrator dashboard was developed with Next.js [21]. These tools were selected because they allow one TypeScript-based workflow to cover server code, mobile screens, and web dashboard components while still leaving each platform with its own deployment target.

a) Version Control and Build Tools:

The source code is managed with Git. The backend is arranged as a NestJS workspace containing the API gateway, authentication service, and reports service. Each backend application has a Dockerfile, and Docker Compose starts the backend services together with RabbitMQ and Redis for local development [18]. For deployment, the workflow builds Docker images for the gateway, auth service, and reports service before running them on Azure App Service [19].

b) Front-end Development:

The mobile app uses Expo, React Native, and TypeScript [5, 6, 4]. React Navigation controls movement between authentication, citizen, relief, report, and profile screens. Redux Toolkit stores authentication state [29]. Expo SecureStore is used when the user chooses to keep the login after closing the app. Device location and image picker APIs support flood report creation because a report usually needs both a place and visible evidence.

The web dashboard uses Next.js, React, TypeScript, and Tailwind CSS [21, 22, 4, 23]. Its main audience is the administrator. The dashboard loads reports from the backend, formats report status values, displays evidence thumbnails, filters reports, and sends status changes through the API gateway.

c) Back-end Development:

The backend follows a microservices style using NestJS microservice support [12]. The API gateway exposes REST endpoints and the Socket.IO gateway [17]. The auth service owns registration, login, profile loading, profile updates, refresh tokens, logout, and user administration. The reports service owns report creation, report retrieval, report update, report deletion, evidence metadata, and cache refresh logic.

RabbitMQ carries messages from the gateway to the backend services [13]. Authentication requests are sent to the auth queue, while report requests are sent to the reports queue. The gateway adds timeout and retry behavior so client requests can fail in a controlled way when a service is unavailable.

d) Database and Media Storage:

PostgreSQL is used for durable data: users, refresh tokens, and reports. TypeORM maps these tables into service entities [14]. Redis stores selected report cache entries so repeated report lookups do not always need to read from PostgreSQL [16]. Cloudinary stores report evidence files [8]. The database keeps only the media metadata, such as public URL, public ID, and resource type, using JSONB fields where the report needs flexible evidence data [15].

e) Cloud Service and Runtime Configuration:

The deployed backend uses Azure App Service as its runtime target. Sensitive values are not written directly into the source code. They are passed through environment variables, including database URL, Redis settings, RabbitMQ credentials, JWT secrets, refresh-token secret, Cloudinary credentials, and administrator seed values. This separation lets the same codebase run locally and in the deployed environment with different configuration values.

Table 4.1: Main implementation tools used in VietFlood.

Area	Tools and implementation use
Backend	NestJS, TypeScript, TypeORM, RabbitMQ, Socket.IO, JWT, bcrypt, and Docker.
Shared library	<code>vietflood_common</code> for logging, Redis access, and Cloudinary operations.
Database and cache	PostgreSQL for stored records and Redis for report cache entries.
Media storage	Cloudinary for uploaded report evidence and generated media URLs.
Mobile app	Expo, React Native, React Navigation, Redux Toolkit, SecureStore, image picker, and device location.
Web dashboard	Next.js, React, TypeScript, Tailwind CSS, token storage, and admin report review.
Deployment	Docker Compose, Docker Hub, GitHub Actions, and Azure App Service.

4.2 Messaging Broker Implementation

RabbitMQ is used as the communication layer between the public API gateway and the internal backend services [13, 12]. The gateway receives HTTP requests from mobile and web clients, prepares the request payload, and sends it to the service that owns the business logic. The auth service listens on `auth_queue`. The reports service listens on `reports_queue`.

The gateway does not directly import report or authentication logic. Instead, it sends message patterns such as `register`, `sign_in`, `profile`, `refresh`, `logout`, `create`, `get_all_by_users`, `update`, and `delete`. This keeps the gateway as a transport boundary and keeps each service responsible for its own data rules.

Gateway calls are wrapped with timeouts and retries. Upload-heavy requests, such as report creation and report update, are allowed more time because they may involve Cloudinary. Smaller queries use shorter limits. When a backend service returns an error or cannot respond, the gateway turns that failure into a controlled HTTP response instead of leaving the mobile or web client waiting.

Table 4.2: Comparison of message brokers for the VietFlood backend.

Criteria	RabbitMQ	Redis Pub/Sub	Kafka
Request-response work	Suitable for gateway commands, queues, and acknowledgements.	Fast, but weaker for durable command handling.	Powerful, but too heavy for this prototype.
Operational cost	Simple to run locally with two backend services.	Simple, but already used for cache in VietFlood.	Requires more setup and topic management.
VietFlood usage	Used for auth and report service communication.	Used separately as the report cache layer.	Not used in the current system.



Figure 4.1: RabbitMQ communication flow between the API gateway, auth service, reports service, auth queue, and reports queue.

4.3 Authentication and Authorization Implementation

VietFlood uses username-password authentication with short-lived access tokens and refresh tokens. User passwords are hashed with bcrypt before they are stored [25]. After a successful login, the auth service signs an access token and creates a refresh token. The access token follows the JSON Web Token format [26]. Its payload carries the user ID, username, role, first name, and last name, which allows the gateway to apply role checks on protected routes.

4.3.1 Authentication Mechanism Implementation

a) Registration Process

Registration is handled by the auth service. Before creating a user, the service checks whether the submitted email, username, or phone number is already used. If any of these values already exists, the request is rejected. If the values are available, the password is hashed using bcrypt with ten salt rounds, and the new record is saved in the `users` table.

The mobile registration screen sends the fields required by the user entity, including email, username, password, phone number, name fields, date of birth, and address information. The backend returns a small success response after creation. It does not send the stored password hash or the full database record back to the client.

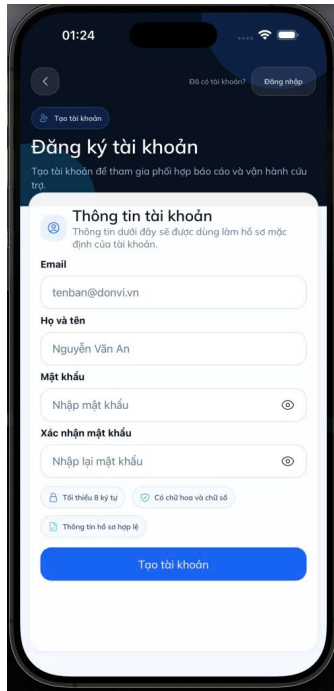


Figure 4.2: Citizen registration screen in the VietFlood mobile app.

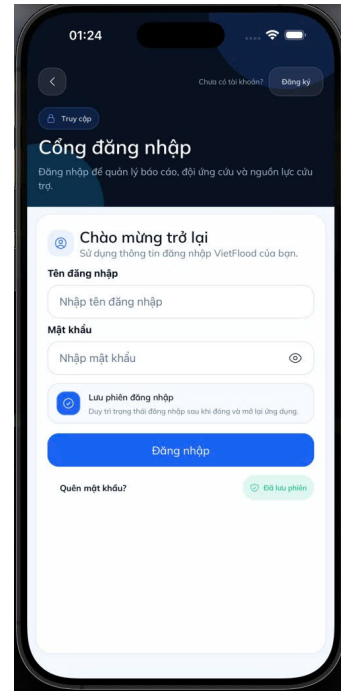


Figure 4.3: Citizen login screen in the VietFlood mobile app.

b) Login Process

Login is exposed through the `/auth/sign_in` endpoint. The auth service searches for the user by username, compares the submitted password with the stored bcrypt hash, and rejects invalid credentials. If the credentials are valid, the service returns an access token and a refresh token.

The mobile app and web dashboard use the login response in different ways. On mobile, the user can choose between a saved login and a session-only login. A saved login writes tokens to SecureStore, while a session-only login keeps them in memory. On the web dashboard, tokens are stored in browser storage, and the dashboard loads the current profile before showing admin-only pages. If the loaded profile is not an administrator profile, the dashboard does not allow entry.

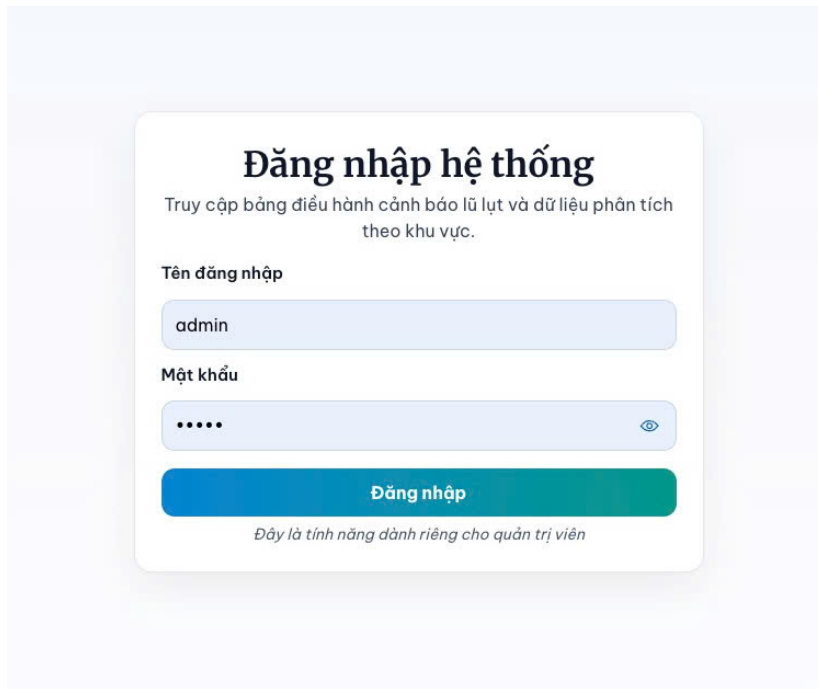


Figure 4.4: Admin login page in the VietFlood web dashboard.

4.3.2 Refresh Token Implementation

Refresh tokens are stored as hashed values in the `refresh_tokens` table. Each row is connected to one user and includes expiration and revocation information. When a user signs in again, the refresh-token service can update the existing row instead of creating an unlimited number of active tokens for the same account.

The refresh endpoint lets the mobile app and web dashboard request a new access token without asking the user to log in again. On mobile, the token storage helper keeps the same storage behavior that was selected at login time. A session-only login remains session-only after refresh, while a saved login writes the refreshed values back to SecureStore.

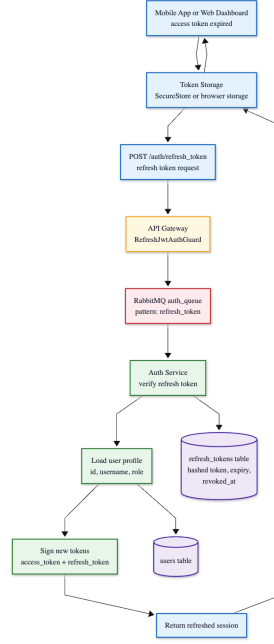


Figure 4.5: Refresh-token flow from mobile or web client through the API gateway, auth service, refresh-token table, and new access token response.

4.3.3 Authorization Process Implementation

Authorization is checked at the API gateway. JWT guards validate the token, and role guards decide whether the route can be used by a citizen, relief user, or administrator. Citizens can create reports and manage their own submitted reports. Relief users can read reports and inspect report locations for response work. Administrators can review evidence, update report status, and manage users.

This permission split is important in VietFlood. Relief users are not responsible for verifying or rejecting reports in the current system. Their screens support field awareness, such as report lists, report details, and map views. Verification, rejection, and resolution remain administrator actions.

4.4 Report Processing and Evidence Upload Implementation

Report creation begins in the mobile application. The citizen fills in category, description, address, coordinates, urgency, severity, and optional evidence images. The app sends this data as multipart form data so text fields and image files can travel in the same request.

The API gateway receives the files in memory and uploads them to Cloudinary under the report evidence folder [8]. It then sends the cleaned report payload and evidence metadata to the reports service through RabbitMQ. The reports service stores the report in PostgreSQL. Category values are kept as arrays, and evidence is stored as JSONB values containing the public URL, public ID, and resource type [15]. After saving, the service writes a Redis cache entry for the report ID [16].

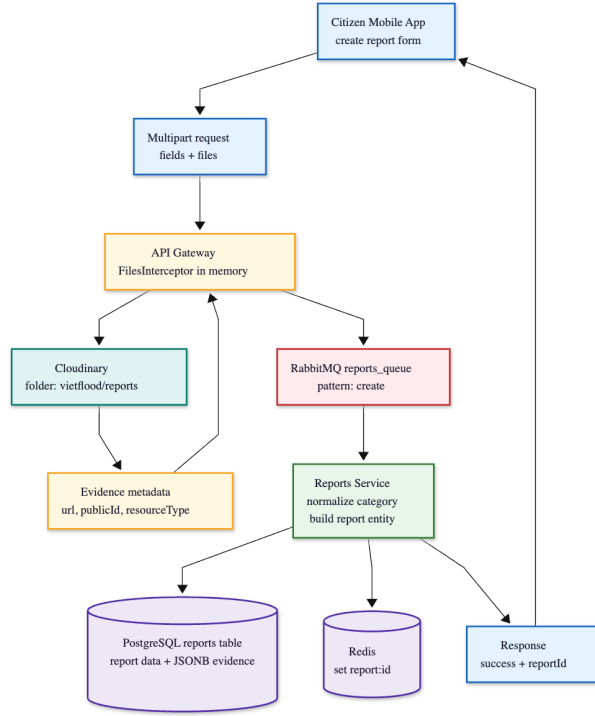


Figure 4.6: Report creation flow from mobile form submission to Cloudinary upload, RabbitMQ message, PostgreSQL insert, and Redis cache write.

4.4.1 Report Update and Delete Implementation

Report update reuses the same upload path as report creation. If the citizen attaches new evidence, the gateway uploads the files to Cloudinary and adds the returned metadata to the update payload. The reports service checks that the report belongs to the user, merges new evidence with the old evidence, updates image URLs, removes the outdated Redis cache entry, and writes a fresh cache entry.

Report deletion also checks ownership before changing data. The reports service reads Cloudinary public IDs from the stored evidence metadata and calls the shared bulk-delete helper when media exists. It then deletes the report row from PostgreSQL and removes the related Redis cache key.

4.5 Public Library Implementation

The shared backend package is implemented as `vietflood_common`. It contains reusable infrastructure services for logging, Redis, and Cloudinary. Without this package, the gateway, auth service, and reports service would need to repeat the same connection and helper logic.

The Cloudinary helper supports buffer upload, file-path upload, base64 upload, single delete, bulk delete, rename, optimized image URL generation, and raw URL generation. In the report workflow, the gateway uses buffer upload for evidence files, while the reports service uses bulk delete when a report is removed.

The Redis helper wraps common operations such as `set`, `get`, `del`, and `publish` [16]. VietFlood mainly uses it for report cache records. The logger service adds service names and request context to logs, which makes backend behavior easier to trace while running several services at the same time.

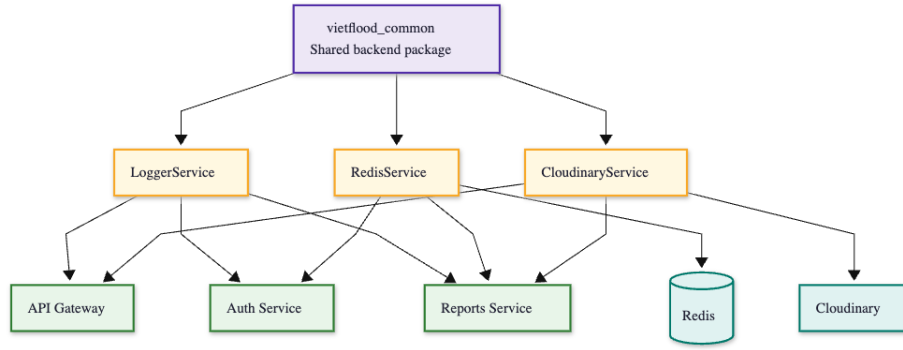


Figure 4.7: Shared common library structure showing LoggerService, RedisService, CloudinaryService, and the backend modules that import them.

4.6 Real-Time Tracking Implementation

VietFlood includes a Socket.IO gateway for live location sharing [17]. The client sends a location event with latitude, longitude, accuracy, heading, speed, and timestamp. The gateway validates the latitude and longitude before broadcasting. Invalid coordinates return a location error to the sender, while valid coordinates are sent to connected clients.

The gateway also handles socket connection and disconnection events. When a client disconnects, other connected clients can be informed that the user is no longer available. This feature supports response-oriented scenarios because a relief screen can receive a citizen's latest location while the report is being handled. In the current prototype, the implementation proves the live communication path; authenticated sockets and stored location history can be added later.

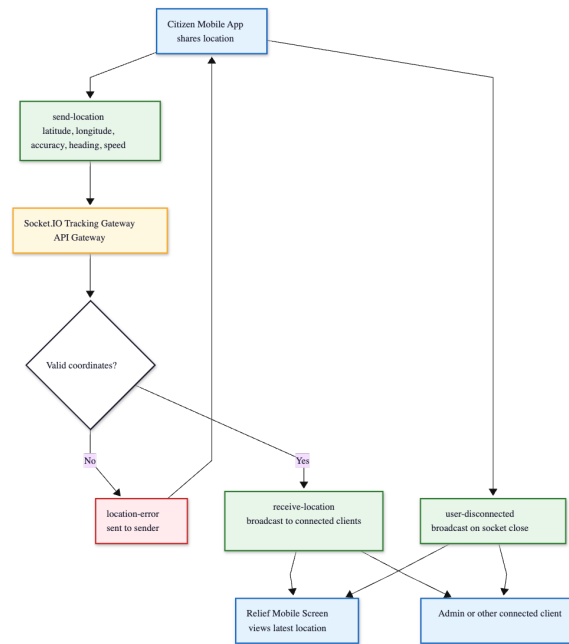


Figure 4.8: Socket.IO live tracking flow with send-location, receive-location, location-error, and user-disconnected events.

4.7 Mobile Citizen Feature Implementation

4.7.1 Report List and Profile Access

The mobile app provides the main citizen workflow after login. It loads the user profile through an authenticated endpoint and retrieves reports through the report service wrapper. The wrapper normalizes backend data before rendering because some fields may appear in different naming styles across the backend and client models.

The citizen report list displays submitted reports with status, category, location, and evidence information. Status labels are normalized before display so the interface remains stable when the backend returns older or alternate status values. This makes the report list more tolerant while the prototype is still evolving.

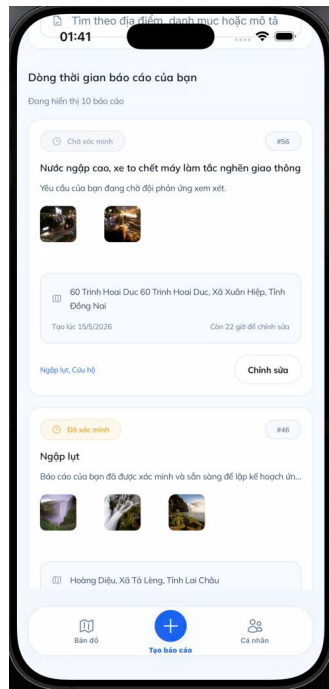


Figure 4.9: Citizen report list or citizen home screen showing submitted reports, status labels, and report location information.

4.7.2 Create Report Implementation

The create-report screen collects the information needed for a useful flood report: category, description, address, latitude, longitude, urgency, severity, and evidence images. Province and ward selection uses data from the Vietnam Provinces Open API [7]. If the user grants location permission, the mobile app can fill coordinate values from the device. The final request is sent as multipart form data so the backend receives both report fields and files.

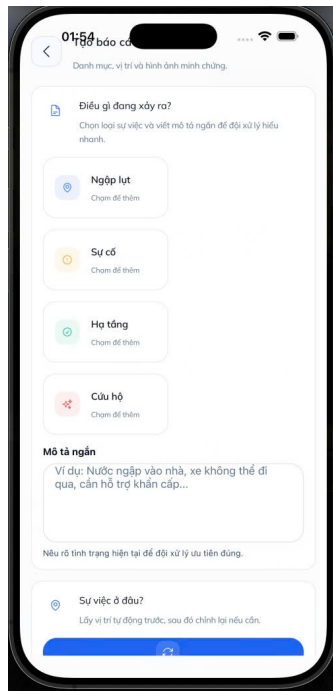


Figure 4.10: Create report form in the VietFlood mobile app.

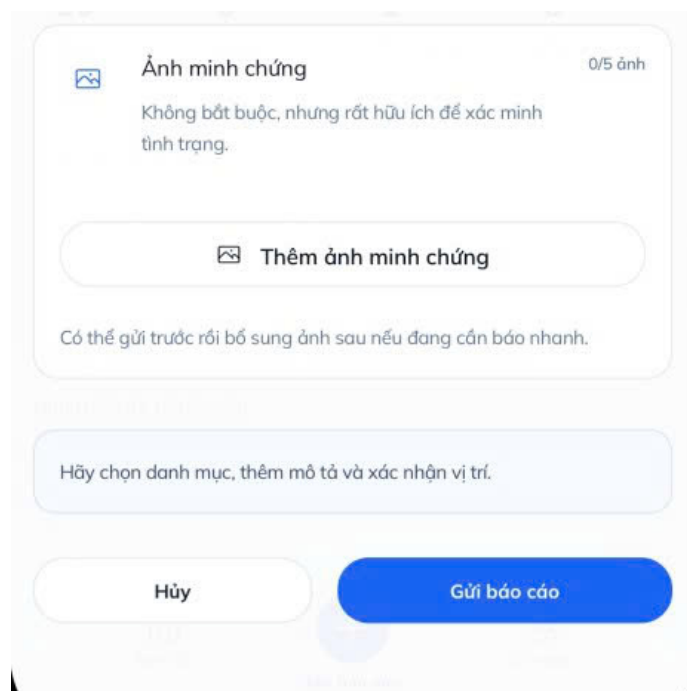


Figure 4.11: Evidence upload in the VietFlood mobile app.

4.8 Relief Feature Implementation

The relief side of the mobile app is built for reading and locating flood reports. The relief report list shows submitted cases, and the detail view presents description, category, address, evidence, and current status. The map screen helps response users understand where reports are placed geographically.

The relief role is deliberately narrower than the administrator role. Relief users can

use report and map information for response coordination, but they do not verify, reject, or resolve reports. The mobile app follows this rule by focusing relief screens on viewing, navigation, and situational awareness.

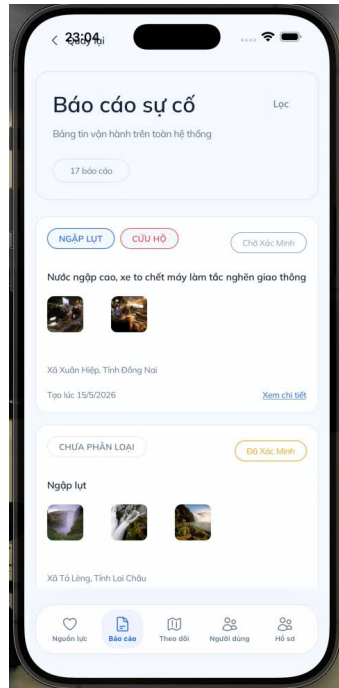


Figure 4.12: Relief report list in the mobile app.

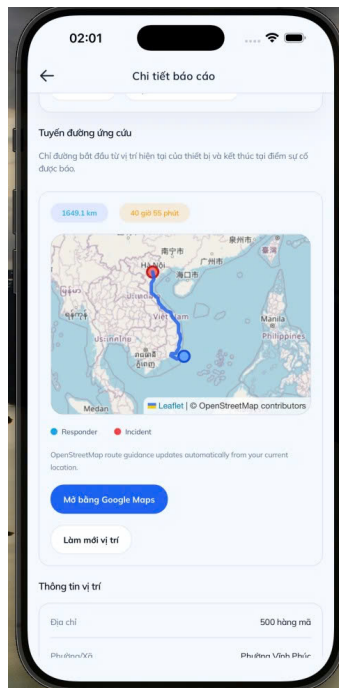


Figure 4.13: Relief map screen in the mobile app.

4.9 Web Dashboard Implementation

The web dashboard is the administrator workspace. It starts from an admin login screen, stores the returned tokens, and loads the current profile before displaying protected pages.

The reports overview loads report records from the backend, formats dates for Vietnamese display, translates category and status values, extracts thumbnail images from evidence, and shows reporter information when it is available.

The dashboard includes search and status filtering. Search can match reporter name, account name, phone number, location, and description. Status filtering supports pending, verified, resolved, and rejected states. The dashboard also provides status controls, which call the admin report update route through the API gateway.

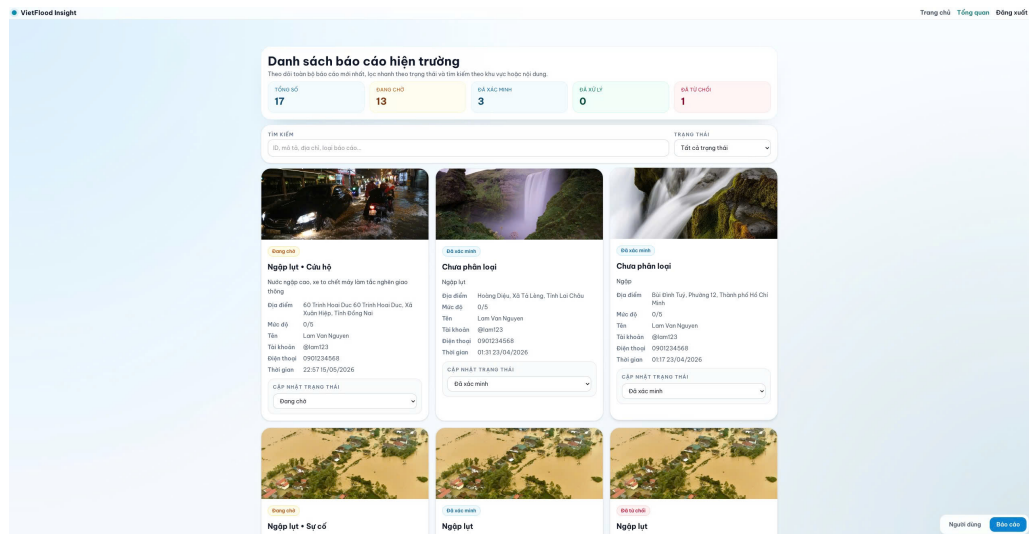


Figure 4.14: Admin dashboard overview with report statistics, search, status filters, report cards, and evidence thumbnails.

4.9.1 Evidence Review and Status Update

Evidence review uses the Cloudinary URLs stored with the report. The dashboard first looks for the evidence list and then falls back to the image list. This fallback makes the interface more tolerant of reports created by different client versions. When the administrator changes a report status, the dashboard sends the update to the gateway. The gateway then forwards the request to the reports service through RabbitMQ.

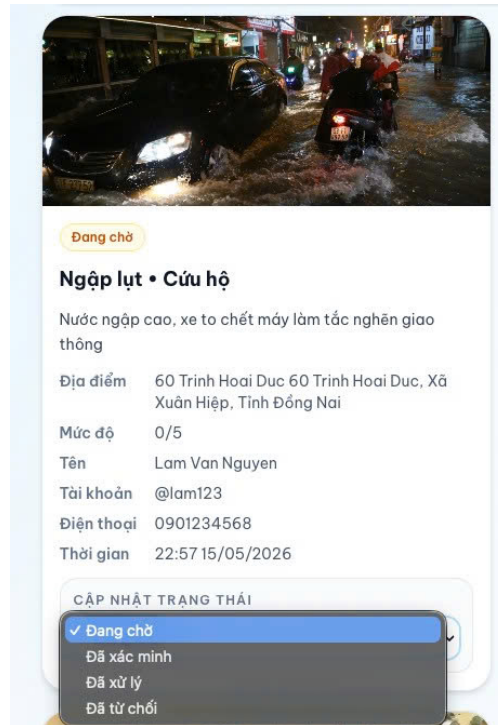


Figure 4.15: Admin evidence review and status update screen for pending, verified, resolved, and rejected reports.

4.10 Deployment Implementation

The backend deployment is based on Docker images and a multi-container Azure App Service setup [19]. In local development, Docker Compose builds the API gateway, auth service, and reports service from source and starts RabbitMQ and Redis beside them [18]. In production, the same service structure is kept, but the backend applications are pulled as Docker Hub images.

The GitHub Actions workflow builds images for the gateway, auth service, and reports service [20]. After the images are pushed to Docker Hub, the workflow configures Azure App Service with the Compose file and restarts the application. This makes deployment repeatable and keeps the release process close to common continuous delivery practice [27, 28].

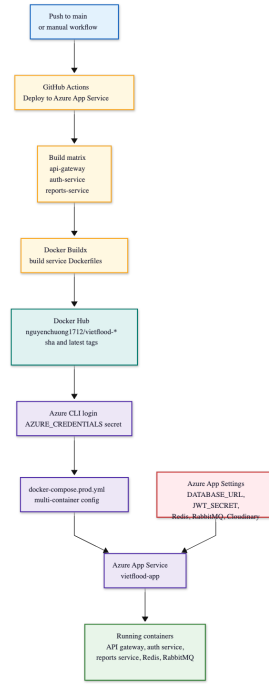


Figure 4.16: CI/CD workflow from GitHub push to Docker image build, Docker Hub push, Azure App Service configuration, and application restart.

4.11 User Management Implementation

User management is implemented in the auth service and exposed through the API gateway. The service can list users, load one user by ID, update a profile, update a user by ID, and delete a user. All roles share the same user table, with the role stored as an enum value for citizen, relief, or administrator.

The web dashboard uses profile validation before showing admin-only screens. Mobile profile screens use authenticated profile and update endpoints so users can maintain their own information. The role model is intentionally simple for this prototype: roles are stored in the database, and route access is enforced by guards in the gateway.

ID	Họ Tên	username	EMAIL	Vai trò	Ngày tạo
3	System Admin	admin	admin@vietflood.com	Quản trị viên	10/03/2024
5	Lam Van Nguyen	lam23	lam@gmail.com	Người dân	14/03/2024
6	Minh Quang Tran	minhtran99	minhtran@gmail.com	Người dân	14/03/2024
7	Huong Thi Le	huong188	huong@gmail.com	Người dân	14/03/2024
8	Tuan Anh Pham	tuapham01	tuapham@gmail.com	Người dân	14/03/2024
9	Linh Thi Do	linhdo22	linhdo@gmail.com	Người dân	14/03/2024
11	Thao Ngoc Ho	thaoho	thaoho@gmail.com	Người dân	14/03/2024
12	Viet Duc Le	vietdu77	vietduc@gmail.com	Người dân	14/03/2024
13	Han Van Tran	hantran	hantran@gmail.com	Người dân	14/03/2024
14	Trang Thi Pham	trangtham	trangtham@gmail.com	Người dân	14/03/2024
15	Hieu Duc Nguyen	hieu	hieuyn@gmail.com	Người dân	14/03/2024
17	Phuong Thi Do	phuongdo	phuongdo@gmail.com	Người dân	14/03/2024
18	Vinh Quang Huang	vinhhuang	vinhhuang@gmail.com	Người dân	14/03/2024
19	Dung Tien Vu	dungtv	dungtv@gmail.com	Người dân	14/03/2024
20	Thanh Minh Bui	thanhbui	thanhbui@gmail.com	Người dân	14/03/2024
21	Ngô Thị Ngô	ngongo	ngongo@gmail.com	Người dân	14/03/2024
22	Son Ngoc Dang	sondang	sondang@gmail.com	Người dân	14/03/2024
28	Huy Duc Ho	huyho6	huyho6@gmail.com	Người dân	14/03/2024
30	Linh Thi Ngo	linhngo8	linhngo@gmail.com	Người dân	14/03/2024
31	Minh Tien Ph	minhtienph	minhtienph@gmail.com	Người dân	14/03/2024

Figure 4.17: User management view showing citizen, relief, and admin accounts with profile and role information.

Chapter 5

Evaluation

This chapter evaluates the VietFlood prototype against the aims of the project and the implementation presented in Chapter 4. The evaluation is based on the available source code and the implemented workflows in the backend, mobile application, web dashboard, and shared library. The main concern is whether the system supports the essential flood-reporting path: a citizen submits a report, evidence and location are preserved, administrators can review the report, and relief users can use report information for response awareness.

The evaluation is limited to prototype-level checking. VietFlood has not yet been tested in a real flood response operation with citizens, administrators, and relief teams working under live conditions. Therefore, this chapter does not claim measured reductions in response time, public adoption, or emergency-service performance. It focuses on what can be observed from the implemented system and what still needs stronger testing before the project can be considered production-ready.

5.1 Evaluation Scope

The evaluation follows the three roles supported by VietFlood. A citizen should be able to create an account, sign in, submit flood reports with location and evidence, and view submitted reports. A relief user should be able to read reports and use location-oriented screens for response work. An administrator should be able to enter the web dashboard, inspect reports and evidence, update report status, and manage user accounts.

The backend is evaluated through its service boundaries. The API gateway acts as the public entry point for REST requests and Socket.IO events. The authentication service is responsible for users, login, profile data, refresh tokens, logout, and user management. The reports service is responsible for report records, evidence metadata, status values, report cache handling, and deletion. RabbitMQ connects the gateway to the services. PostgreSQL stores durable data. Redis supports report caching. Cloudinary stores uploaded evidence files.

The mobile and web clients are evaluated separately because they serve different users. The mobile application, built with Expo React Native, covers citizen and relief workflows [5, 6]. The web dashboard, built with Next.js, supports administrator review [21]. This separation matches the expected use of the system: citizens and relief users are more likely to work from phones, while administrators need a dashboard for checking incoming reports and changing report status.

5.2 Functional Evaluation

The first functional question is whether a citizen report can travel through the whole system without losing its important information. In the implemented prototype, that flow exists. The mobile report form collects category, description, address, coordinates, urgency, severity, and optional evidence files. The gateway receives the multipart request, uploads evidence to Cloudinary, and sends the report payload to the reports service

through RabbitMQ. The reports service stores the record in PostgreSQL and refreshes the Redis cache for the saved report.

The second question is whether the stored report can be reviewed by the right user. The admin dashboard reads reports from the same backend. It can display status, reporter information, location fields, urgency, severity, and evidence thumbnails. It also allows administrators to update report status values such as **pending**, **verified**, **resolved**, and **rejected**. This creates the main review loop of VietFlood: citizen submission, backend storage, administrator review, and status update.

Table 5.1: Functional evaluation of VietFlood user flows.

Scenario	Expected behavior	Evaluation result
Citizen account access	A citizen can register, sign in, and load profile data after authentication.	The auth service supports registration, sign-in, profile loading, and profile updates. Passwords are hashed with bcrypt, and protected requests use JWT access tokens.
Citizen report creation	A citizen can submit a flood report with location, description, urgency, severity, and optional evidence.	The mobile app prepares the report request, the gateway receives multipart files, Cloudinary stores evidence, and the reports service saves report data with evidence metadata.
Citizen report history	A citizen can view reports connected to the signed-in account.	The backend provides a user-report retrieval path, and the mobile app normalizes report fields such as coordinates, address, category, and status before display.
Admin report review	An administrator can inspect reports and update their status after checking evidence.	The dashboard loads report records, supports status filtering, displays evidence thumbnails, and calls the admin update route for status changes.
Relief report access	A relief user can view reports, open details, and work with map-based report information.	Relief screens focus on report awareness and location support. In the current permission model, relief users cannot verify or reject reports.
Token refresh and logout	A signed-in user can renew a session and can end it when needed.	The backend provides refresh-token and logout flows. The mobile app keeps the user's selected storage mode when refreshing tokens, while the dashboard refreshes tokens from browser storage.
Live location events	A client can send live coordinates, and connected clients can receive valid updates.	The Socket.IO gateway checks latitude and longitude ranges before broadcasting. Invalid coordinates produce a location error instead of being sent to other clients.

The relief role deserves a separate note. Some mobile screens show status-related report information, but the right to verify, reject, or resolve a report stays with administrators. This is a reasonable boundary for the prototype. Relief users need fast access to report

details and map information, while status approval affects the trustworthiness of the whole platform.

5.3 Backend Evaluation

The backend structure is suitable for the size and purpose of the prototype. The API gateway, auth service, and reports service separate the two main business areas: identity management and flood report management. This supports the architectural goal of separating responsibilities so the system is easier to maintain and modify [24]. The gateway handles client-facing concerns such as HTTP routes, file upload parsing, JWT guards, role checks, and Socket.IO events. The services keep data rules and service-specific logic behind RabbitMQ message patterns.

RabbitMQ is useful because the report workflow is not a single database insert [13]. A report can include evidence files. The gateway uploads those files to Cloudinary, builds evidence metadata, and sends the final payload to the reports service. The reports service then writes PostgreSQL data and updates Redis. Timeout and retry handling in the gateway gives the client a controlled failure response when a service cannot answer in time.

PostgreSQL fits the current data model. Users, refresh tokens, and reports all have structured fields. Reports also need flexible evidence metadata, and JSONB is used for data such as evidence URL, public ID, and resource type [15]. This keeps the prototype simpler than a design with a separate evidence table, while still preserving the information needed to display and delete uploaded media.

Redis is used in a limited and understandable way [16]. The reports service writes report cache entries after create and update operations, and deletes cache entries when reports are changed or removed. This is enough for the current prototype. If VietFlood later needs heavy map queries, province filters, or large report lists, the cache layer should be expanded beyond individual report keys.

The shared package, `vietflood_common`, is also a positive implementation choice. It keeps Cloudinary helpers, Redis helpers, and logging helpers in one place. The gateway can use shared upload utilities, while the reports service can use the shared deletion utilities. This avoids repeating infrastructure code across services without mixing the business logic of authentication and reporting.

5.4 Client Evaluation

The mobile application is the most important client because flood reports are usually created in the field. The implemented app covers account access, profile screens, citizen report creation, report lists, evidence upload, device location support, and relief-oriented report views. It also uses the Vietnam Provinces Open API for province and ward selection, which makes address entry closer to Vietnamese administrative data.

The mobile token flow is practical for different usage situations. A user can keep tokens in Expo SecureStore for persistence, or keep them only for the current session. When the app refreshes a token, it respects the same storage decision. This is useful for people who may use a shared device or do not want the app to remain signed in after closing.

The web dashboard has a narrower purpose, and that is appropriate. It is not meant to replace the mobile app. Its job is administrator review. The dashboard loads backend reports, formats report data, shows evidence thumbnails, filters by status, displays reporter

details where available, and sends status updates through the API gateway. It also loads the current profile after login so non-admin users cannot enter the admin workspace.

The main weakness in the clients is field consistency. The mobile app has to normalize values such as `lat/lng` and `latitude/longitude`, and it also handles different naming styles between backend and client models. This makes the app more tolerant during development, but it also suggests that the API contract should be made stricter before a wider release.

5.5 Security and Access Control Evaluation

VietFlood applies role-based access through the API gateway. JWT guards validate access tokens, and role guards separate citizen, relief, and admin actions. Citizens can create and manage their own reports. Relief users can read report data for response work. Administrators can update status values and manage users.

Password storage uses bcrypt hashing [25]. Access tokens use JWT claims to carry the user ID, username, role, first name, and last name [26]. Refresh tokens use a separate secret and are stored with expiration and revocation information. This gives the backend a way to end sessions and reject expired refresh tokens.

Some security work should be strengthened before production use. The refresh-token path should be tested carefully, especially hashing, comparison, revocation, and rotation behavior. The Socket.IO tracking feature validates coordinate ranges, but it should also be more tightly connected to authenticated users and role rules before being used during real response activities.

Evidence upload also needs stronger protection. Clouinary stores the uploaded media, but VietFlood should still enforce file count, file size, file type, and scanning rules at the gateway level. Public upload features can create spam, storage-cost, and abuse risks if limits are not applied.

5.6 Deployment and Operations Evaluation

The deployment setup is suitable for prototype demonstration. Local development can run the gateway, auth service, reports service, RabbitMQ, and Redis through Docker Compose [18]. The production workflow builds Docker images for the backend services, pushes them to Docker Hub, and deploys the multi-container application to Azure App Service [20, 19].

Runtime configuration is handled through environment variables. The database URL, Redis settings, RabbitMQ credentials, JWT secrets, refresh-token secret, Clouinary credentials, and administrator seed values are not written directly into the source code. This is important because the same service images can be run with different settings in local and deployed environments.

The largest operational gap is monitoring. The shared logger gives the services a consistent logging style, but a real deployment would need more than logs. Health checks, alerting, error dashboards, upload failure tracking, and rollback steps would make the system easier to operate. In a flood reporting context, administrators need to know quickly when reports stop arriving, uploads fail, or a backend service becomes unavailable.

5.7 Evaluation Result

The evaluation shows that VietFlood satisfies the core prototype requirements. A citizen can submit a flood report with location and evidence. The backend can store the report, preserve evidence metadata, and expose the report for review. Administrators can inspect reports and change their status from the web dashboard. Relief users can view report information and use mobile screens that support location awareness.

The system is still a prototype, not an operational disaster-response platform. It needs user testing, load testing, slower-network testing, stronger upload limits, stricter tracking authorization, clearer API contracts, and better monitoring. The current code demonstrates that the main software workflow is feasible. It does not yet prove that the system will remain reliable during a real flood event with many users and unstable network conditions.

For the thesis scope, the result is acceptable. VietFlood demonstrates a multi-platform flood monitoring and reporting system using a NestJS microservice backend, a React Native mobile client, a Next.js admin dashboard, PostgreSQL, Redis, RabbitMQ, Cloudfire, Docker, and Azure deployment. The next evaluation step should move from implementation review to controlled trials with real users, real devices, and larger report volumes.

Chapter 6

Conclusion and Future Works

6.1 Conclusion

This thesis presented VietFlood, a multi-platform flood reporting prototype for citizen reporting, relief awareness, and admin review. The project began from a practical problem: flood information often reaches communities through scattered messages, photos, and location shares. That information can help, but it is hard to organize when many reports arrive at the same time. VietFlood gives that process a clearer path. A citizen can submit a report from the mobile app, attach evidence, provide location details, and let the backend store the report for later review.

The completed prototype connects four main parts. The NestJS backend provides the API gateway, authentication service, reports service, and live tracking gateway. The Expo React Native mobile app handles citizen and relief workflows. The Next.js web dashboard handles admin login, report filtering, evidence preview, and status updates. The shared `vietflood_common` package keeps Redis, Cloudinary, and logging utilities in one place so the backend services do not repeat the same infrastructure code.

The implementation shows that the main report loop works as a software system. The API gateway accepts report submissions and evidence files. Cloudinary stores uploaded media. PostgreSQL stores users, refresh tokens, and report data. Redis stores report cache entries. RabbitMQ moves backend requests from the gateway to the auth and reports services. Socket.IO handles live location updates with coordinate validation. This design follows the microservices pattern of separating independent capabilities into owned services while using message brokers for communication [9, 11, 10]. These parts do not solve every disaster-response problem, but they prove that the prototype can move a citizen report from mobile submission to backend storage and admin review.

Role separation is one of the clearer outcomes of the system. Citizens create and manage their own reports. Relief users read report data and use mobile screens focused on response awareness. Admin users review reports and update report status. This separation keeps verification and rejection under admin control, while relief users can still inspect reports and locations for field work. That boundary is useful because report status affects trust in the system.

The project also shows the value of building the system across mobile, web, backend, and shared code rather than treating flood reporting as only a mobile form. The mobile app is close to the citizen and relief user. The web dashboard gives admins a larger view for review. The backend holds the rules, storage, messaging, and role checks. The shared library keeps repeated infrastructure code under control. Together, these parts make VietFlood a base for later testing and improvement.

At the same time, VietFlood should be understood as a prototype. It has not yet been tested in a real flood situation, with many users, unstable network connections, and urgent field decisions. The current system demonstrates the technical path, not full operational readiness. Before practical use, it needs field testing, stronger upload rules, better monitoring, stricter live-tracking authentication, and more careful review of token behavior.

6.2 Future Works

Future work should begin with field testing. The system needs feedback from citizens, relief workers, and administrators using real phones and realistic network conditions. That testing would show whether the report form is clear, whether evidence upload is reliable enough, and whether the admin dashboard provides enough context for status review.

The mobile app should also support poor-network conditions better. A future version should let citizens save reports locally, retry submission when the connection returns, and compress evidence files before upload. The app should also make the report state clear: sent, saved, uploading, or failed.

Report review can be improved through duplicate detection, spam handling, and evidence quality checks. Reports with similar coordinates, descriptions, and creation times could be grouped for admin review. Relief workflows could also add assignment, response notes, and route history while keeping verification and rejection under admin control.

Security and operations should be stronger before public deployment. Refresh-token handling, file size limits, file type checks, live-tracking authentication, audit logs, health checks, service alerts, and deployment rollback steps all need more work. The operational work should follow DevOps and deployment automation practices [28, 27]. If report storage or evidence upload fails, administrators should know quickly through monitoring and alerting. The containerized deployment using Docker Compose is already a base, but observability, alerts, and recovery steps are needed before production use [18].

In later research, VietFlood could also add flood insight analysis. The existing reports already contain category, urgency, severity, coordinates, address fields, status, and timestamps. With enough verified reports, the system could generate simple heat maps, time-based charts, or province-level summaries. These additions should come after the reporting path is stable. Bad input will produce bad insight, so data quality must come first.

Bibliography

- [1] M. F. Goodchild and J. A. Glennon, “Crowdsourcing geographic information for disaster response: a research frontier,” *International Journal of Digital Earth*, vol. 3, no. 3, pp. 231–241, 2010.
- [2] M. Imran, C. Castillo, F. Diaz, and S. Vieweg, “Processing social media messages in mass emergency: A survey,” *ACM Computing Surveys*, vol. 47, no. 4, pp. 67:1–67:38, 2015.
- [3] NestJS, “Nestjs documentation.” <https://docs.nestjs.com/>, 2026.
- [4] TypeScript, “Typescript documentation.” <https://www.typescriptlang.org/docs/>, 2026.
- [5] Expo, “Expo documentation.” <https://docs.expo.dev/>, 2026.
- [6] Meta Open Source, “React native documentation: Introduction.” <https://reactnative.dev/docs/getting-started>, 2026.
- [7] Vietnam Open API, “Province open api.” <https://provinces.open-api.vn/>, 2026.
- [8] Cloudinary, “Upload api reference.” https://cloudinary.com/documentation/image_upload_api_reference, 2026.
- [9] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term.” <https://martinfowler.com/articles/microservices.html>, 2014.
- [10] J. Thönes, “Microservices,” *IEEE Software*, vol. 32, no. 1, p. 116, 2015.
- [11] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*, pp. 195–216, Springer International Publishing, 2017.
- [12] NestJS, “Microservices documentation.” <https://docs.nestjs.com/microservices/basics>, 2026.
- [13] RabbitMQ, “Consumer acknowledgements and publisher confirms.” <https://www.rabbitmq.com/docs/3.13/confirm>, 2026.
- [14] TypeORM, “Typeorm documentation: Getting started.” <https://typeorm.io/docs/getting-started/>, 2026.
- [15] PostgreSQL Global Development Group, “Postgresql documentation: Json types.” <https://www.postgresql.org/docs/current/datatype-json.html>, 2026.
- [16] Redis, “Redis documentation.” <https://redis.io/docs/latest/>, 2026.
- [17] Socket.IO, “Socket.io documentation.” <https://socket.io/docs/v4/>, 2026.
- [18] Docker, “Docker compose documentation.” <https://docs.docker.com/compose/>, 2026.

- [19] Microsoft, “Continuous deployment with custom containers in azure app service.” <https://learn.microsoft.com/en-us/azure/app-service/deploy-ci-cd-custom-container>, 2026.
- [20] GitHub, “Github actions documentation.” <https://docs.github.com/en/actions>, 2026.
- [21] Vercel, “Next.js documentation.” <https://nextjs.org/docs>, 2026.
- [22] React, “React documentation.” <https://react.dev/>, 2026.
- [23] Tailwind Labs, “Tailwind css documentation.” <https://tailwindcss.com/docs>, 2026.
- [24] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 4th ed., 2021.
- [25] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *1999 USENIX Annual Technical Conference*, USENIX Association, 1999.
- [26] M. B. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” RFC 7519, Internet Engineering Task Force, 2015.
- [27] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [28] G. Kim, P. Debois, J. Willis, and J. Humble, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016.
- [29] Redux Toolkit, “Redux toolkit documentation.” <https://redux-toolkit.js.org/>, 2026.

Appendix A

API Specifications

This appendix provides representative API response structures for the VietFlood system. Use recorded Postman responses or logged API traffic to replace placeholders where available.

A.1 Authentication API

Endpoint: POST /auth/sign_in

Response:

```
1 {
2   "statusCode": 200,
3   "message": "Login successful",
4   "data": {
5     "accessToken": "<JWT_ACCESS_TOKEN>",
6     "refreshToken": "<JWT_REFRESH_TOKEN>",
7     "user": {
8       "id": "<uuid>",
9       "email": "<user_email>",
10      "role": "citizen"
11    }
12  }
13 }
```

Listing A.1: Sign In Response

A.2 Report Creation API

Endpoint: POST /reports

Response:

```
1 {
2   "statusCode": 201,
3   "message": "Report created successfully",
4   "data": {
5     "id": "<report_id>",
6     "category": ["flood"],
7     "description": "<description>",
8     "status": "pending",
9     "evidence": [
10      {
11        "url": "<cloudinary_url>",
12        "resourceType": "image"
13      }
14    ]
15  }
16 }
```

Listing A.2: Create Report Response

Appendix B

Interface Screenshots

Key user interface screens are shown here. These images support the main report without repeating the implementation details.

B.1 Mobile Screens

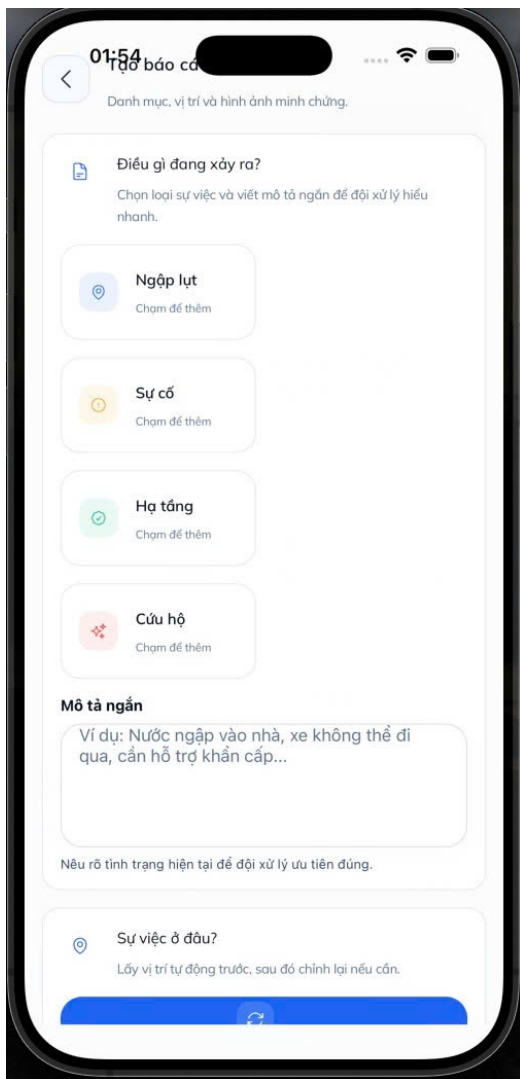


Figure B.1: Citizen report creation screen.

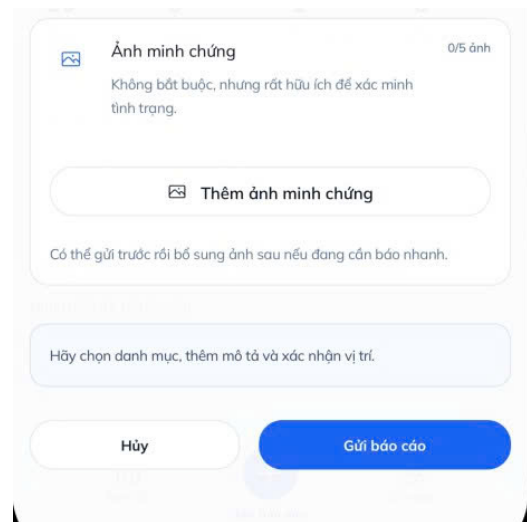


Figure B.2: Evidence upload screen on the mobile app.

B.1.1 Relief and Admin Screens

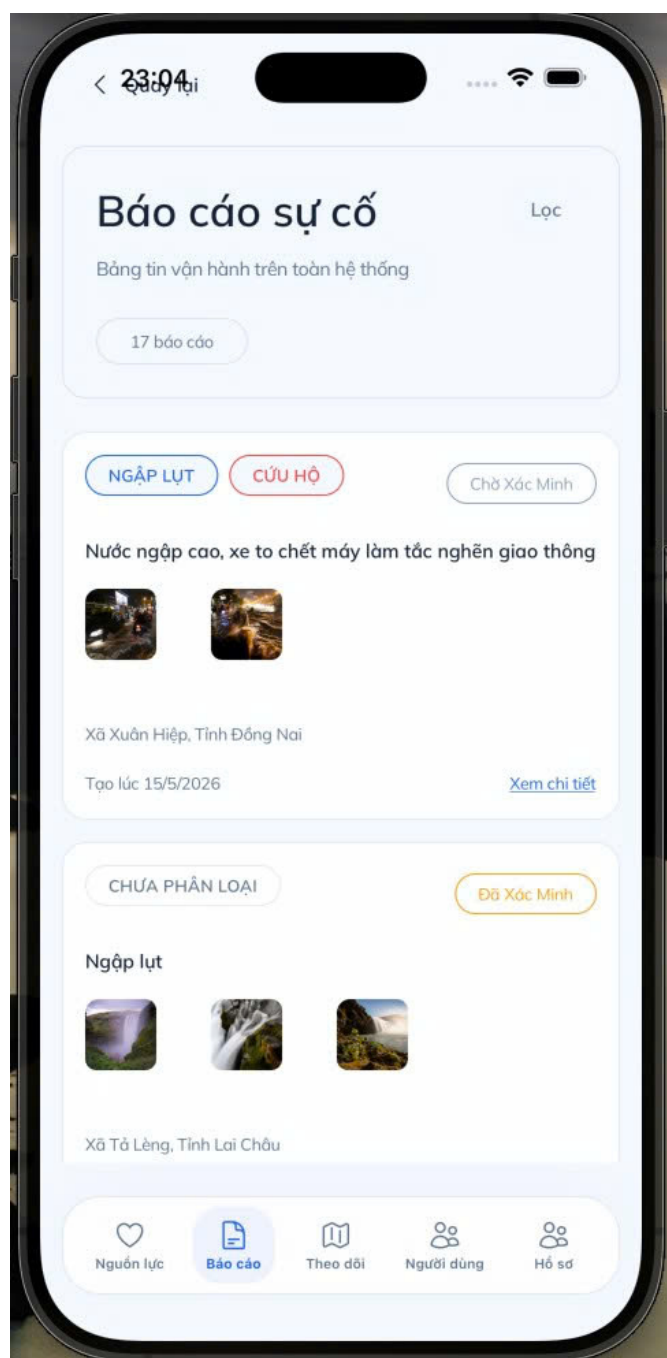


Figure B.3: Relief worker report list screen.

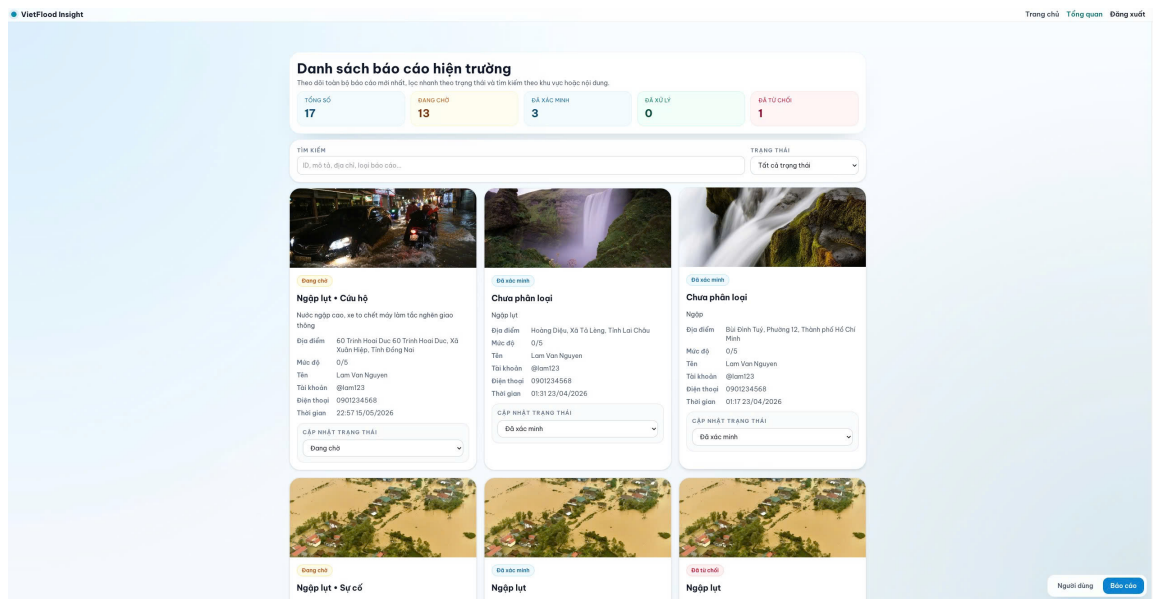


Figure B.4: Admin dashboard overview.

Appendix C

Test Cases

This table summarizes the most important validation flows for the VietFlood prototype.

Table C.1: Key VietFlood Test Cases

Scenario	Expected Result	Status
User login	Access token and user profile are returned	To be completed
Create report with evidence	Report saved and evidence links stored	To be completed
View user reports	Citizen can view own report list	To be completed
Admin verify/reject report	Report status updates correctly	To be completed
Relief user views map	Map shows report locations and markers	To be completed
Refresh token flow	New access token issued without login	To be completed

Appendix D

Deployment and Configuration

This chapter summarizes the deployment configuration groups used by VietFlood. Sensitive values are masked and should be provided through secure environment variables.

Table D.1: Deployment Configuration Groups

Group	Example Variables	Notes
Database	POSTGRES_HOST POSTGRES_PORT POSTGRES_DB POSTGRES_USER_NAME POSTGRES_PASSWORD DATABASE_URL	Stores PostgreSQL connection settings. Real values must be masked.
JWT	JWT_SECRET REFRESH_SECRET	Stores access-token and refresh-token secrets.
RabbitMQ	RABBITMQ_DEFAULT_USER RABBITMQ_DEFAULT_PASS RABBITMQ_URL	Stores message broker connection settings.
Redis	REDIS_HOST REDIS_PORT REDIS_PASSWORD REDIS_DB	Stores cache service configuration.
Cloudinary	CLOUDINARY_CLOUD_NAME CLOUDINARY_API_KEY CLOUDINARY_API_SECRET	Stores media upload configuration.

D.1 Deployment Notes

- Use masked values for secrets in the thesis and replace them with actual values in deployment.
- Store credentials in CI/CD secrets or environment configuration, not in source control.
- Change default RabbitMQ credentials before production use.
- Set JWT secrets as cryptographically secure strings.